Evaluating the Impact of Augmenting the VAEP Model with 360-Degree Freeze-Frame Spatial Context

Candidate Number: 291723

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur convallis leo sit amet euismod finibus. Nullam vestibulum metus nec mauris varius, eu aliquam purus commodo. Sed faucibus ex nisl, id bibendum ligula ultrices nec. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla volutpat gravida sem, et consequat diam pretium ac. Donec condimentum pulvinar scelerisque. Orci varius natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Morbi elementum felis sapien, id euismod metus tincidunt vel. Mauris elementum eu neque elementum molestie. Morbi accumsan, nunc in malesuada semper, neque urna suscipit neque, non ornare velit purus id massa. Sed ut mi vel leo egestas euismod vel sit amet orci. Sed sed placerat odio. Mauris tincidunt id ex id commodo. Etiam hendrerit lacus et velit gravida, in volutpat enim aliquet. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia curae; Cras vel tortor sagittis, dignissim purus sit amet.

Table of Contents

1. Introduction 4
2. Research Objectives 5
3. Background 5
3.1. Valuing Actions by Estimating Probabilities (VAEP) 5
3.2. Related Work: Applications of Spatial Context to Action Value Metrics 7
3.2.1. Expected Goals (xG) 7
3.2.2. Expected Threat (xT) 7
3.2.3. Expected Possession Value (EPV) 8
3.2.4. Summary 9
3.3. Representing Spatial Context 9
3.3.1. Manual Spatial Feature Engineering 9
3.3.2. Spatial Embeddings: Set Transformer 9
4. Methodology 11
4.1. Datasets Outline 11
4.1.1. Limitations of Datasets 12
4.2. Data Collection 13
4.3. Data Preprocessing 13
4.4. Data Engineering 13
4.4.1. Manually-Engineered Spatial Context Features 13
4.4.2. Spatial Context Embeddings 14
4.5. Set Transformer Hyperparameter Tuning 17
4.5.1. Tree-Structured Parzen Estimatior (TPE) 18
4.5.2. Hyperband Pruning 20
4.6. Classifier Training 20
4.6.1. Logistic Regression 21
4.6.2. Random Forest 21
4.6.3. Gradient Boosting 22
4.6.4. Loss Function: Cross-Entropy Loss 22
4.6.5. Split Criterion: Gini impurity 22
4.7. Model Evaluation Metrics 23
4.7.1. Area Under the Receiver Operating Characteristic Curve (AUC-ROC) 23
4.7.2. Brier Score 23
4.7.3. Cross-Entropy Loss 24
4.7.4. Expected Calibration Error (ECE) 24
4.8. AUC Difference Significance Testing 24
4.9. Model Interpretability 25
4.9.1. Permutation Importance 25
4.9.2. Partial Dependence Plots (PDPs) 25
4.9.3. k-Means Clustering 25
4.9.4. Uniform Manifold Approximation and Projection (UMAP) 28
5. Results 29
5.1. Set Transformer Training 29
5.1.1. K-means Clustering: Spatial Embeddings 33
5.1.2. UMAP 38
5.1.3. Spatial Embedding Dimension and Manually-Engineered Feature Correlations . 40
5.2. Classifier Training 41
5.2.1. Feature Set Significance Tests 43
5.2.2. Classifiers Used for VAEP in this Project 46
5.2.3. Valuing Actions by Estimating Probabilities (VAEP) EDA 46

5.3. Model Interpretation 49
5.3.1. Permutation Importance 49
5.3.2. Partial Dependence Plots 55
6. Discussion 58
7. Conclusion 58
References 59

1. Introduction

The adoption of analytics in association football has grown significantly in the last couple of decades (Goes et al., 2021; Olthof & Davis, 2025; Schaerlaeckens, 2025; Wakelam et al., 2022), enabling football clubs to leverage insights to inform tactical decisions, coaching and recruitment. Much of this innovation has involved the development of metrics used to determine the value provided by a particular action made by a player in terms of goal-scoring probabilities. One such metric is Valuing Actions by Estimating Probabilities (VAEP) (Decroos et al., 2019).

VAEP considers the probability of scoring or conceding within the next k events to assign a value between -2 and 2 to each action. VAEP, like other such metrics, typically uses action-centric features such as the action's start and end location and type (carry, pass, shot) in estimating scoring/conceding probabilties, but this approach does not explicitly model the wider spatial context (Sá-Freire, 2025). The spatial context - the position of teammates and opponents around the actor - of an action determines the options available to a player, showing viable passing options or space to carry into, as well as the proximity of opposition defenders applying pressure or blocking options.

The StatsBomb Open Data repository (StatsBomb, 2026a) provides events (e.g. carries, passes, shots) data for matches, along with *three-sixty* freeze-frames which indicate the positions of players within proximity of the acting player. Such data can be used to incorporate spatial context as a predictive feature. To illustrate this, **Figure 1** compares a pass event from a match with and without the surrounding spatial context. Without the spatial context, this pass appears to offer limited progression towards the opponent's goal. Traditional VAEP would consider how passes of this trajectory tend to result in scoring/conceding, but the addition of spatial context differentiates this particular pass from others of similar trajectory. **Figure 1** shows how the left hand side of the actor is blocked by multiple opponents (red) and this pass moves the ball into space on the right flank to open an attacking opportunity (in this case, a long-ranged shot)[1]. Spatial context could allow finer-grained differentiation, rather than aggregating all passes of this trajectory.

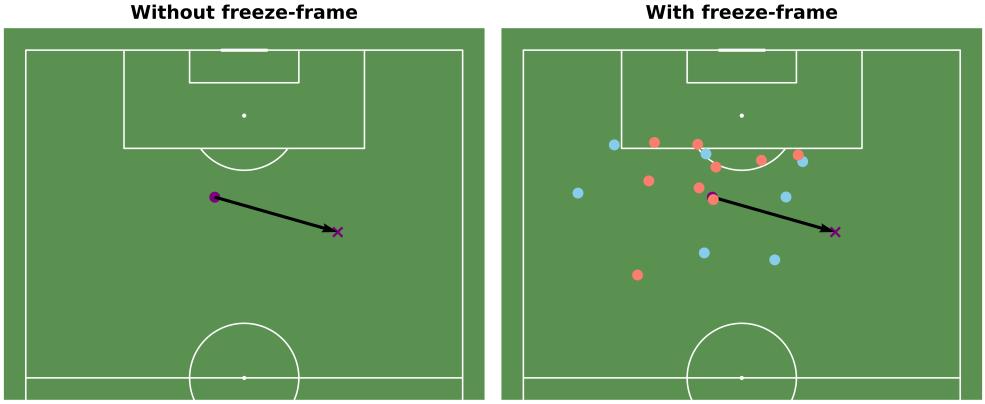


Figure 1: Pass Event With and Without Freeze-Frame

[1]The pass event id is '23b6b26a-293c-4d5e-9889-5a5ce0b7ad8c' and the subsequent long-ranged shot event id is 'd9d27d20-35bc-42bc-ac53-059ad519f781'

2. Research Objectives

The general aim of this project is to determine to what extent the addition of spatial context features enhances and alters the standard VAEP model. This entails the following objectives:

- Develop representations of spatial context from StatsBomb 360 freeze-frame data using feature engineering and deep learning techniques.
- Train and evaluate VAEP models, both with and without the inclusion of spatial context representations, using supervised classification algorithms.
- Compare and interpret these VAEP models using appropriate peformance metrics and an exploration of feature importance.

3. Background

In soccer analytics, *actions* refer to a *type of event* within a match which is caused by a player (e.g. a pass), with *events* being the broader set of significant occurrences in a match (e.g. passes, but also substitutions, fouls etc.) (Decroos, Bransen, Van Haaren, & Davis, 2019). Typical performance metrics focus on action outcomes, such as goals, assists and pass completion rates. Whilst such metrics can be used to determine the value of some actions and are useful for summarising player ability, soccer analytics has developed metrics to help generalise this notion of action value. Moreover, models used to assign value to actions have been enhanced by incorporating the locations of teammates and opponents, leading to improved performance and interpretability.

As such, this section will introduce the VAEP metric and outline how spatial context has been successfully incorporated into other action valuation metrics, motivating this project's research objectives.

3.1. Valuing Actions by Estimating Probabilities (VAEP)

Decroos, Bransen, Van Haaren, & Davis (2019) introduce Valuing Actions by Estimating Probabilities (VAEP). VAEP values actions in terms of the probability of scoring and the probability of conceding within the next k actions. Equation 1 defines the VAEP of an action a_i for team x.

$$V(a_i, x) = \Delta P_{scores}(a_i, x) - \Delta P_{concedes}(a_i, x), \quad V(a_i, x) \in [-2, 2] \tag{1}$$

If we consider a match as a sequence of m on-ball actions $[a_1, ..., a_m]$, we can view some game state S_i which corresponds to some subset of actions $[a_1, ..., a_i]$ and determine how the last action in the game state a_i changed the probability of team x scoring and conceding within the next k actions. The change in the probability of scoring (and conceding) from the previous game state $S_{i-1} = [a_1, ..., a_{i-1}]$ to the current game state S_i can be determined by simply finding their difference:

$$\begin{aligned} \Delta P_{\text{scores}}(a_i, x) &= P_{\text{scores}}(S_i, x) - P_{\text{scores}}(S_{i-1}, x) \\ \Delta P_{\text{concede}}(a_i, x) &= P_{\text{concede}}(S_i, x) - P_{\text{concede}}(S_{i-1}, x) \end{aligned} \tag{2}$$

These two probabilities are estimated using two supervised classification algorithms, one which estimates scoring, and the other conceding, within the next k actions. The feature sets used across these two models are identical, all that varies is the labels (scoring or conceding). Decroos, Bransen, Van Haaren, & Davis (2019) used a feature set which corresponds to the

Soccer Player Action Description Language (SPADL) format involving 9 features, most notably the action's start and end locations, and the type of action. The authors also add "complex features", such as the time elapsed between consecutive actions, as well as "game context features", such as the scoreline at the time of the action and the amount of time remaining.

Decroos, Bransen, Van Haaren, & Davis (2019) initially compared logistic regression, a random forest, XGBoost and CatBoost, finding CatBoost to perform best across both the scoring and conceding models in terms of the Brier Score and the Area Under the Receiver Operating Characteristic Curve (AUC-ROC). Decroos, Bransen, Van Haaren, & Davis (2019) also extend the VAEP metric by using it to provide player ratings, where the VAEP of each action in a player's set of actions A_p^T across some time frame T are considered cumulatively, and normalised per-90 minutes played during T.

$$\text{rating } (p) = \frac{90}{m} \sum_{a_i \in A_p^T} V(a_i) \tag{3}$$

The authors compare these VAEP player ratings against traditional metrics like goals and assists per-90, as well as market valuations of players. The authors found the top-10 highest players ranked by VAEP captured players typically considered to be amongst the best in the Premier League, despite them not appearing in the top-10 goals + assists per-90 ranking. The top-10 VAEP ranked list also had a higher overall market valuation than the top-10 goals + assists per-90 ranked list (€$1,110$ Million vs $947 Million$), which suggests VAEP more comprehensively represents player value compared to traditional metrics.

Decroos & Davis (2020) extend their previous work by improving model interpretability. Given that the initial findings from Decroos, Bransen, Van Haaren, & Davis (2019) suggest the use of gradient boosting algorithms for VAEP model classifiers, these models are more difficult to interpret than methods such as logistic regression. As such, Decroos & Davis (2020) built a generalised additive model (GAM) on the top-10 most important features from the original VAEP feature set, as determined by the average *gain* (for the present classifiers, this is the reduction of Gini impurity - see Section 4.6.5) that feature imports on the model across all splits involving that feature. The authors found a very small drop in classifier performance: a 0.2% increase in the normalised Brier score compared to XGBoost on the original VAEP feature set, and a 0.1 increase on the top-10 most important feature set. The authors argue the large gains in utility came from the added interpretability from being able to assess each univariate function which comprised their GAM model. This enables a direct observation of how each function contributes to its prediction across the domain of that function. For example, *distance to opponent goal* constituted one of the top-10 features, and figure 2 in Decroos & Davis (2020) shows how the goal scoring probability changes across the domain of this feature's function. Matching intuition, a higher goal scoring probability is observed for lower distances to the opponent's goal, with this probability dropping exponentially as distance increases. This added interpretability helps to demonstrate the greater tactical understanding which soccer analysts may benefit from when using a GAM-based VAEP model, and that such interpretability may be gained without a significant loss in performance.

One potential limitation of the VAEP model is that it lacks a representation of wider spatial context from the positions of teammates and opponents during actions (Sá-Freire, 2025), due to its use of event data only. Every action taken by a player exists within a spatial context which determines the options available to a player, influencing their decision making. The use of spatial context in VAEP models is also understudied, despite its addition having been successfully applied to a number of alternative action value metrics.

3.2. Related Work: Applications of Spatial Context to Action Value Metrics

3.2.1. *Expected Goals (xG)*

Expected goals (xG) is an approach commonly used for assigning value to individual shots (Green, 2012; Macdonald, 2012). General to xG approaches is the representation of the value of some shot as the probability that it results in a goal. These goal probabilities are estimated via various classification algorithms such as logistic regression, gradient boosting and deep learning methods (Mead et al., 2023). Features used in xG models typically include *action-intrinsic* variables such as distance/angle to the centre of the opponent's goal and the body part used, along with *match-context* variables such as the current match scenario (e.g. set piece, open play) (Anzer & Bauer, 2021; Mead, O'Hare, & McMenemy, 2023; Robberechts & Davis, 2020).

Cefis & Carpita (2025) incorporate spatial context into the xG model by considering the number of players ahead of the shooter (x co-ordinate greater than the shooter's) within a fixed radius (y co-ordinate $\pm 10\%$ of the shooter's). From this count, they derive two binary variables `D1.OpponentsPlayer` and `D2.OpponentsPlayer`, where the former equals 1 when the number of players in this area is greater than 0, and the latter equals 1 when the number of players in the area is greater than 1. They also consider the x and y co-ordinates of the opponent's goalkeeper (`GK x coordinate`, `GK y coordinate`). Table 3 in Cefis & Carpita (2025) show the coefficients and odds ratios of each feature in their logistic regression xG model across their 4 different techniques for addressing class imbalance. `D1.OpponentsPlayer`, `D2.OpponentsPlayer`, and `GK x coordinate` all have negative coefficients, indicating a reduction in the log odds of the shot resulting in a goal. This is particularly pronounced for `D2.OpponentsPlayer`, with a coefficient range of -6.13 to -5.69, indicating that the probability of scoring (i.e. xG) is reduced by around 99.73% (because $1 - e^{(-5.69 + -6.13)/2} \approx 0.9973$) when there is more than 1 opposition defender within proximity of the shooter. This finding matches intuitions about the influence of the opposition's positioning on goal probability and highlights how the addition of spatial context during modelling can enhance the xG metric.

3.2.2. *Expected Threat (xT)*

A further action value metric is expected threat (xT). xT measures the value of actions based on where on the pitch the action moves the ball from and to. Singh (2018) derived the following equation for calculating the xT of a given location on the pitch:

$$\mathrm{xT}_{x,y} = \left(s_{x,y} \times g_{x,y}\right)\left(m_{x,y} \times \sum_{z=1}^{Z}\sum_{w=1}^{W} T_{(x,y)\to(z,w)}\mathrm{xT}_{z,w}\right) \tag{4}$$

If we discretise the football pitch into some $Z \times W$ (Singh (2018) uses $Z = 16$ and $W = 12$) set of equally-sized square zones (x, y), we can consider three possible attacking options when the ball is in some zone (x, y): shoot, carry or pass. $\left(s_{x,y} \times g_{x,y}\right)$ quantifies the value of shooting from zone (x, y) by weighting the probability of scoring $g_{x,y}$ by the probability of shooting $s_{x,y}$. The other two options, carrying or passing, involve moving the ball from (x, y) to (z, w). The transition matrix $T_{(x,y)}$ provides the probablity of moving the ball from zone (x, y) to any zone (z, w) after 1 time step. If we consider all $Z \times W$ possible options and multiply their transition probabilities by the value of its respective (z, w) zone, we can take their sum and weight it by the probability of actually moving the ball to another zone when in (x, y). This provides a quantification of passing or carrying the ball, which when multiplied by the shot-probability weighted xG of zone (x, y) provides the xT of the ball being at zone (x, y). However, given that Equation 4 assumes that the value of each zone $\left(\mathrm{xT}_{z,w}\right)$ is already known a-priori, we must

estimate its value. The xT model thus begins by setting every zone (x, y) to have an xT of 0. This simplies Equation 4 to essentialy be the xG of zone (x, y), weighted by how likely players are to actually shoot from that zone:

$$\mathrm{xT}^{(0)}_{x,y} = \left(s_{x,y} \times g_{x,y}\right) \tag{5}$$

We can then take the xT values from this first iteration and recalculate them iteratively until convergence. Singh (2018) suggests 4–5 iterations are sufficient but that this may vary across datasets. To then assign xT to an action, we simply subtract the xT of the end location of the action (i.e. the xT of zone (z, w)) by the xT of the start location (the xT of zone (x, y)). This approach is useful for allowing us to determine the value of any on-ball attacking action, not just shots. We can also quantify genuinely valuable actions at any point during a chain of actions (e.g. an incisive pass during a sequence which ends in a goal), rather than suffering from a bias towards teminating actions (goals, assists).

Everett et al. (2022) introduce spatial context to the xT transition matrix $T_{(x,y)}$ via a pitch control (Spearman et al., 2017) and interception model (Burriel & Buldú, 2021) which was used to train a convolutional neural network (CNN) to produce the transition probabilities for their contextual transition matrix $T^{\star}_{(x,y)}$. The pitch control model considers the probability of a player controlling the ball as a function of their distance from opposition players, whilst the interception model introduces an additional transition probability, that being the probability of the ball being intercepted when moving from zone (x, y) to its intended destination (z, w). Everett, Beal, Matthews, Norman, & Ramchurn (2022) found the introduction of spatial context yielded a modest improvement in terms of model fit, reducing the log likelihood of the xT model from -5.137 to -4.151. Greater gains were found in its useful applications to tactical analysis, such as enabling the authors to apply a Basin-Hopping optimiser (Wales & Doye, 1997) to find more optimal defensive shapes than which actually occurred during actions, using their derived Threat Above Expected (TAx) metric as the algorithm's loss function.

3.2.3. *Expected Possession Value (EPV)*

Expected possession value (EPV) (Fernandez et al., 2021) is an action valuation metric, derived from other sports such as basketball (Cervone et al., 2016), which exploits tracking data to assign value to a game state (e.g. during an action) at time t via a consideration of all possible subsequent paths it can take.

$$\mathrm{EPV}_t = \sum_{a \in A} \mathbb{E}[G \mid A = a, T_t] \quad \mathbb{P}(A = a \mid T_t) \tag{6}$$

Equation 6 shows that the EPV at time t for some action is determined by the expected value of G (typically a goal being scored or not) given some action a has occurred in some spatiotemporal context T_t, multiplied by the probability of that action a occurring in the spatiotemporal context T_t. For each action a considered in the action set A, we sum these expected values to provide an EPV of some game state at time t. Thus EPV can be applied to any game state, not simply at the time at which discrete match events or actions occur.

We can see from the term T_t that EPV relies upon spatial context to operate. Fernandez, Bornn, & Cervone (2021) list a number of spatial features used in the EPV model, such as pitch control, number of players within 3m of the ball location, and the number of players within the area circumscribed by the ball and the two posts of the opponent's goal. EPV is thus inherently sensitive to spatial context in its valuation of actions. The utility provided by EPV's

incorporation of spatial context is explored by Fernandez, Bornn, & Cervone (2021). For example, they use EPV to determine the on-ball (passes with positive EPV) and off-ball (pitch locations with positive EPV if a pass were to arrive there) advantage Liverpool in 2014/15 had during possession when up against different opponent formations. This analysis shows both where Liverpool were successful and unsuccessful in their passing decision (on-ball) and also where valuable regions of the pitch are to direct passes towards during build-up (off-ball). Such an analysis is extremely useful for tactical decision making as, in response to opposition pressing structures, teams can evaluate where on the pitch they should look to progress the ball towards to be most effective during build-up play, and to evaluate how successful their current approach is.

3.2.4. *Summary*

To summarise, the discussed applications demonstrate the utility provided by incorporating spatial context into action valuation models. Given the lack of investigation into augmenting VAEP models with wider spatial context, this project addresses this research gap and explores what role representations of spatial context play in altering model behaviour and, potentially, performance.

3.3. Representing Spatial Context

3.3.1. *Manual Spatial Feature Engineering*

Previous research has represented spatial context using features engineered from spatial data.

As described in Section 3.2.1, Cefis & Carpita (2025) derived two binary features `D1.OpponentsPlayer` and `D2.OpponentsPlayer` from player x and y co-coordinates, indicating the number of players within the immediate frontal proximity of the shooter.

Peters et al. (2024) developed an Expected Pass Turnovers (xPT) metric, used to measure the probability that a given pass would result in a turnover (conceding possession to the opposition) which can be used to assess how player's passing ability during some time interval compares to what would be expected on average, given the set of passes made by that player and the contextual factors relating to those passes. The authors derive xPT values from a binary logistic mixed-effects model trained on a feature set which included two features derived from StatsBomb *three-sixty* freeze-frames: "three pressure radii" (the number of opponents within three radii of varying, but unspecified length around the ball carrier) and number of "unmarked teammates" (teammates with no opponents within 2m of them). The authors found the addition of these "positional" features improved model AUC by 6.1 points

Amichay et al. (2025) measured the areas of convex hulls and inner layers to represent team shapes. A convex hull, in this context, is the area inscribed between the outer-most positioned players on the pitch for a given team. A larger area corresponds to a wider/longer team shape, whereas a smaller area is narrower/shorter. Since this convex hull only considers the outer-most players, a second convex layer can be created from the players who do not contribute to the hull. The ratio of these two areas forms the author's derived Layer Ratio (LR) metric, which summarises the compactness of the team shape.

3.3.2. *Spatial Embeddings: Set Transformer*

An alternative way to represent spatial context is in the form of spatial embeddings. A spatial embedding is a real-valued vector used to represent a set of geometric objects (in this case, player co-ordinates and their characteristics) in a fixed d-dimensional space. These embeddings

can represent non-linear relationships between objects and thus may serve to represent patterns, like defensive and offensive team structures during actions.

However, as seen in Figure 2, StatsBomb *three-sixty* data does not identify individual players within each freeze frame (except from the actor). Thus, to create embeddings for player positions, we must use a neural network which has an input layer which is *permutation invariant*. That is, the output of the network (i.e. the embeddings) should not depend on the order in which features appear in the input layer. This would enable the locations of players in each freeze-frame to be inputted into the network in any order and still produce the same embedding. It would also be useful for such a network to attend to spatial relationships that may exist across freeze-frames, such as defensive shapes.

One useful technique for developing a permutation-invariant network for spatial embeddings is a *set transformer* (Lee et al., 2019). A set transformer is a variation on the standard transformer architecture from Vaswani et al. (2017) which can be used to process sets such that the output of the transformer is invariant to how elements in the set are permuted when entering into the model.

Transformers use multihead self-attention layers in a traditional encoder-decoder architecture. Multihead self-attention works by projecting an input sequence $X \in \mathbf{R}^{n \times d}$ into different learned linear projections of the input embeddings: $\left\{Q = XW_Q, K = XW_K, V = XW_V\right\} \in \mathbb{R}^{n \times d}$. Each row corresponds to a token representation in a different learned subspace. The self-attention mechanism determines how much each token should attend to other tokens, including itself, by computing similarity between so-called *query* vectors and *key* vectors. If the similarity between a *query* and a *key* is high, the corresponding *value* vector contributes more strongly to the output representation. This is computed for all tokens simultaneously, producing pairwise interactions between all elements in the sequence, including self-interactions. The attention mechanism achieves this via scaled dot-product attention:

$$\text{Attention } (Q, K, V) = \text{softmax } \left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{7}$$

Multihead attention then involves performing this scaled dot-product attention in parallel h times, each with their own set of Q, K, V matrices. Outputs are then concatenated and fed into a standard feedforward neural network.

The multi-head attention mechanism of standard transformers is adapted in set transformers via *induced set attention blocks* (ISABs). An ISAB performs self-attention between set elements by introducing *inducing points* which contain parameters learned during training. These inducing points are some fixed-sized vector I and attend to the inputted set X to produce a transformed set H. The input set X then attends to H to produce a final set of n elements. This method improves computational efficiency as the size of the inducing points vector $|I|$ can be set to be smaller than the input size $|X|$, thus avoiding standard pairwise self-attention and reducing time complexity from $O(n^2)$ to $O(nm)$ where $m < n$.

Important for the purposes of this present project, set transformers also enable permutation invariance by dropping position encodings of input set elements and pool outputs via pooling with multihead attention (PMA). PMA works similarly to multihead attention but instead pools the representations of each element of the set into k vectors, each providing a representation of the whole set. k is typically set to 1, provding a single representation of the set, but $k > 1$ is permitted. The queries Q are a fixed set of learned *seed vectors* S. Reordering the set will thus result in the same pooled representation via PMA, enabling permutation invariance.

As such, the encoder component of the set transformer is used to generate spatial embeddings. However, regarding the decoder, this component is simply used during the training process. The decoder takes the encoder's generated spatial embedding as an input and attempts to reconstruct the original freeze-frame array. The deviation between the original array and the decoder's array provides a means to assess how well the encoder's generated spatial embedding represents the freeze-frame array, and thereby constitutes a measure of the set transformer's performance: its loss function. The exact loss function used in this present project is discussed in Section 4.4.2.1. Given that the role of the decoder is to provide a reconstruction objective for evaluating the quality of the encoder's learned spatial embedding, it does not necessarily require the same self-attention mechanisms used by the encoder. Such a design choice to use a lightweight, non-attention based decoder has been applied in the SegFormer architecture (Xie et al., 2021) for regular transformers, and successfully applied to set transformers by Ashcroft et al. (2023) in their work on generating illustrations by training a generative set transformer model to represent them as "stroke clouds" (a type of set comprised of Bézier curves) whilst using a non-attention based diffusion decoder. In this present project, the encoder is responsible for modelling relationships between players and producing a fixed-dimensional representation of the freeze frame, while the decoder acts as a reconstruction head that maps this representation back to the original player freeze-frame. With the decoder playing an instrumental role in the development of spatial embeddings, it is arguably sufficient for the decoder to have a relatively simplistic architecture, provided the loss function used preserves permutation-invariance.

Applications of set transformers in soccer analytics have typically fallen in the domain of trajectory prediction. Kim et al. (2023) apply set transformers in the context of soccer analytics. Kim, Choi, Kim, Yoon, & Ko (2023) use set transformers to encode representations of player trajectories from tracking data snapshots to infer ball trajectories. Capellera et al. (2024) similarly developed *FootBots* using set transformer architecture to improve player and ball motion prediction over traditional models. Whilst this domain deviates from this project, these studies do highlight the appropriateness of using set transformers to encode player positions.

This project seeks to use set transformers to produced fixed-sizes spatial embeddings for each team's set of players within each freeze-frame sample. This can be achieved by first converting each individual co-ordinate pair, along with its characteristics (teammate, goalkeeper, actor), into an embedding via an initial input projection sublayer. Each set of individual co-ordinate embeddings for each team in each freeze-frame can then be passed as an input tensor to the set transformer which will produce embedding representations of each set of co-ordinates. The initial embedding of co-ordinate pairs is necessary to ensure the set transformer has enough dimensions to learn from whereas simply having 5 dimensions (i.e. x, y co-ordinates, and the 3 player attributes) would be insufficient to capture multiple non-linear relationships.

4. Methodology

4.1. Datasets Outline

The project uses data from StatsBomb's Open Data repository. This repository has "made certain leagues of StatsBomb Data freely available for public use for research projects and genuine interest in football analytics" (StatsBomb, 2026a). The StatsBomb Open Data repository provides the following datasets, and data specification documents exist which outline variables, data types and variable descriptions for each dataset:

- **competitions.json:** A single JSON file for competition summary information.
- **events:** A directory containing a single JSON file for each match, detailing each event which takes place within that match (e.g. each pass, shot, carry etc.).

- **lineups:** A directory containing a single JSON file for each match, providing information about each player who participated in that match (e.g. their position, shirt number, nationality).
- **matches:** A directory containing a single JSON file for a season in a given competition, providing summary information about each match played in that season (e.g. match date, home/away teams and scores).
- **three-sixty:** A directory containing a single JSON file for each match, providing data for each event in a given match detailing the position of each player in the broadcaster's frame at the moment of that event.

Decroos, Bransen, Van Haaren, & Davis (2019), along with VAEP, introduce the Soccer Player Action Description Language (SPADL) as a data engineering format to enable "human-interpretable, simple and complete" records of actions within football matches. A variant of this format is used in this project for the baseline feature set, that being Atomic-SPADL (Davis et al., 2020). Atomic-SPADL represents an action without considering its result (success vs failure), unlike SPADL which includes the result. An example of the effect this has on action representation is that a pass (one action under SPADL) is separated into two actions: the giving of the pass, and the reception of the pass. An additional "interception" action type is introduced to enable incomplete passes to be captured. Atomic-SPADL therefore introduces more precision in how actions are represented as actions are now indivisible (hence atomic). This atomic-SPADL format is then augmented in this project by the addition of engineered spatial features.

The project uses *events* and *three-sixty* data corresponding to 292 matches from UEFA Men's Euro 2020 & 2024, UEFA Women's Euro 2022 & 2025, FIFA Men's World Cup 2022, FIFA Women's World Cup 2023. Unlike most competitions in this repository, *three-sixty* data is available for every match in this sample, enabling player positional context to be incorporated into feature engineering for 1,052,116 match events.

4.1.1. *Limitations of Datasets*

Football analysts often makes use of tracking data (Goes, Meerhoff, Bueno, Rodrigues, Moura, Brink, Elferink-Gemser, Knobbe, Cunha, Torres, & Lemmink, 2021). Rather than considering discrete time events, tracking data provides a continuous record of player and ball position for the duration of a match. This enables analysts to model actions in matches as continuous processes, accounting for the position and movements of the entire team, unlike with events data. Such tracking data is often proprietory and is thus not available for public use (Bassek et al., 2025). This limits the scope of the present project to viewing events as discrete in time, thus losing some surrounding context which may be informative (e.g. player movement direction and velocity).

Events data is nonetheless relatively comprehensive, covering a much larger sample of matches than the limited publicly-available tracking data (Olthof & Davis, 2025). Furthermore, the use of StatsBomb's *three-sixty* data does address this limitation somewhat by providing snapshots of proximal player positions at the point of each event, allowing some inference to be made regarding team positioning during match events. However, these snapshots do not necessarily capture every player's location during an event, just those within the view of the broadcast camera (StatsBomb, 2026a).

A further limitation is with the use of SPADL formatted data. Recent work by Jo et al. (2026) has shown that some events data contains logical inconsistencies (i.e. sequences of events which are not logically possible), which the SPADL format does not correct for. The authors develop the Verified Event Data Format for Reliable Soccer Analytics (VERSA) to address this issue

using a state-transition model to automatically identify and correct logical inconsitencies, improving performance on VAEP model training. However, their results on StatsBomb data suggest that the atomic-SPADL format still slightly outperforms VERSA. This result, along with the increased precision in action representation mentioned earlier, motivates the use of the atomic-SPADL format in this project, but it is recognised that the potential data integrity issues highlighted by Jo, Kang, Lee, Lee, Bauer, & Ko (2026) will not be explicitly handeled as implementing VERSA is non-trivial and thus beyond the scope of this project.

4.2. Data Collection

Data was obtained from the StatsBomb Open Data Repository via the `statsbombpy` (StatsBomb, 2026b), `socceraction` (Socceraction Developers, n.d., Decroos, Bransen, Van Haaren, & Davis (2019)), and requests (, n.d.) libraries. The `statsbombpy` and `socceractions` libraries were used to collect raw events data from the matches in each competition, whilst `requests` was used to collect *three-sixty* freeze-frames from the StatsBomb Open Data Repository directly. Events data was then merged with freeze-frame data and each competition's data was saved to an individual parquet file.

4.3. Data Preprocessing

With StatsBomb events and *three-sixty* data collected, the first preprocessing step requires converting these events into SPADL-formatted actions. This was achieved by passing each match's set of events to the `convert_to_actions` function from `socceraction`. This also required the rejoining of freeze frame data, since this preprocessing function only returns SPADL formatted data, not custom features.

To then convert these actions to the atomic-SPADL format, the `convert_to_atomic` function similarly takes each match's set of actions and returns them in atomic-SPADL format. The atomic actions of each match from each competition are then saved to parquet files, one for each competition.

With the derived atomic-SPADL actions, spatial context features are subsequently added during data engineering.

4.4. Data Engineering

4.4.1. *Manually-Engineered Spatial Context Features*

```
{
    "teammate": boolean,
    "actor": boolean,
    "keeper": boolean,
    "location": [float, float]
}
```

Figure 2: StatsBomb *three-sixty* freeze-frame JSON schema

Figure 2 outlines the JSON schema for StatsBomb *three-sixty* freeze-frames. Each freeze-frame JSON object has an associated `event_uuid` to enable joins with events data, and a `visible_area` polygon which indicates the viewing area from the broadcast camera at the moment the action took place. We can see from the available data, that it is possible to construct the opposition

proximity metric from Cefis & Carpita (2025) and a partial convex hull area or LR as explored by Amichay, Silva, Brito, & Marcelino (2025). Given that the sample in this project applies to on-ball actions, not just shots, the opposition proximity metric from Cefis & Carpita (2025) is adapted to consider a 360° radius around the player, rather than a frontal semi-circle.

Thus, for this project, the following spatial features were engineered for teammates and opponents, respectively, using *three-sixty* data. These are both inspired by the literature and also chosen by myself:

- Euclidean distance to nearest teammate (meters)
- Euclidean distance to nearest opponent (meters)
- Number of teammates within 5 meters of the actor
- Number of opponents within 5 meters of the actor
- Average Euclidean distance to teammates in freeze-frame (meters)
- Standard deviation of Euclidean distance to teammates in freeze-frame (meters)
- Average Euclidean distance to opponents in freeze-frame (meters)
- Standard deviation of Euclidean distance to opponents in freeze-frame (meters)
- Teammate convex hull area (based on players within freeze-frame) (m^2)
- Opponent convex hull area (based on players within freeze-frame) (m^2)

To outline the rationale here, opponent spatial metrics are intended to provide a proxy measure of *pressure* applied to the actor and *defensive shape*, which imposes limitations upon where the ball can be progressed to by introducing a risk of losing possession. Teammate spatial metrics provide a proxy measure of *pass availability* for the actor and *rest defense* - the positional structure of a team whilst in possession to prepare for potential counter-attacks (Coaches' Voice, 2024).

For *number of teammates/opponents within 5 meters of the actor*, whilst a radius of 5 meters somewhat arbitrary, Bischofberger et al. (2026) argue that a "lower radius might risk missing actual pressure and involvement while a higher radius might lead to a weaker attribution between players and stronger intra-team correlations" p.5. Calabuig et al. (2024) also use a 5 meter radius around players as part of their construction of *influence areas* - the amount of overlap of two player's circles equates to how much influence the defender has over the attacker, providing a measure of defensive pressure.

It should be noted that such an approach to approximating opposition pressure or teammate pass availability is considered relatively simplistic, with more sophisticated approaches such as pitch control (Fernández et al., 2018) being preferred in the literature. However, given that metrics like pitch control require tracking data to be implemented correctly, they fall beyond this project's scope. Regardless, the intention with these manually-engineered spatial context features is to provide an approximation of spatial context, which is easy to implement, in order to contrast it with the generated spatial context embeddings in terms of classifier performance and behaviour. Given that such a measure has been successfully and usefully incorporated in previous literature, it was deemed appropriate for this present project.

4.4.2. *Spatial Context Embeddings*

To generate an n-dimensional embedding of each freeze-frame for each action, we can begin by converting each JSON object in each freeze-frame to a 1×5 array, using the 5 available features as shown in Figure 2 (the location key can be split into an x co-ordinate feature and a y co-ordinate feature, respectively). Each freeze-frame consists of n players, and so n arrays are stacked into an $n \times 5$ array.

This $n \times 5$ array, constituting each player within a single freeze-frame, is then be projected into a higher dimensional space via an initial learned affine transformation h_i.

$$h_i = W_e x_i + b_e, \quad x_i \in \mathbb{R}^5, h_i \in \mathbb{R}^{d_{\text{hidden}}} \tag{8}$$

The dimensionality d_{hidden} of this learned projection layer is left as a tunable hyperparameter during optimisation, with $d_{\text{hidden}} \in \{32, 64, 128, 256\}$ being the candidates used. This transformation provides the set transformer with a higher-dimensional latent representation of each element, enabling it to learn richer relationships between players within each freeze-frame than would be possible from the raw feature representation alone.

This projection layer $H \in \mathbb{R}^{n \times d_{\text{hidden}}}$ is then inputted to the set transformer encoder, whereby the self-attention mechanism learns the weight matrices W_Q, W_K, W_V which minimise loss (see Section 4.4.2.1).

4.4.2.1. *Set Transformer Loss Function*

Given that the freeze-frames - particularly the player co-ordinates - constitute permutation-invariant sets, the loss function used must also respect this characteristic. Each player in each freeze-frame consists of two broad components: a pair of x, y co-ordinates for their location on the pitch and a set of attributes about that player (teammate, goalkeeper, actor).

For the location component, we require a loss function which preserves the permutation-invariant property of our freeze-frames, which are unordered sets. Chamfer distance fulfills such requirements.

Chamfer distance conducts a nearest neighbour search between points in one set with points in the other set. Each point is assigned its nearest neighbour in the other set and the distance is taken between them. Each of these distances are summed and averaged across the number of elements from the first set. Minimisation of Chamfer distance is achieved by successfully reconstructing the original set. In this scenario, this would correspond to the decoder successfully reconstructing the co-ordinate locations of each player within the original freeze-frame. This nearest neighbour search can be conducted with quadratic time complexity, as we must compute every pairwise distance between original and reconstructed set elements.

$$\mathcal{L}_{\text{Chamfer}} = d_{\text{CH}}\big(S, \hat{S}\big) = \frac{1}{|S|} \sum_{x \in S} \min_{\hat{x} \in \hat{S}} \| \, x - \hat{x} \, \|_2^2 + \frac{1}{\big|\hat{S}\big|} \sum_{\hat{x} \in \hat{S}} \min_{x \in S} \| \, \hat{x} - x \, \|_2^2 \tag{9}$$

Alternative distance measures may be used in place of Chamfer distance, such as the Hungarian method (Kuhn, 1955), which can be used to find an optimal solution to the assignment problem: finding the permutation of the reconstructed set which, when paired one-to-one with the original set, minimises loss. Chamfer loss does not enforce such a one-to-one correspondence, as elements from the original set may share the same nearest neighbour from the reconstructed set (and vice versa).

Rather than searching through each permutation naively, which would result in a factorial search time complexity, the Hungarian method constructs a cost matrix which records the pairwise distances of each point in the original set with each point in the reconstructed set.

To find the optimal permutation, the minimum value of each row is subtracted from its elements. Then, the minimum value of each column is subtracted from each of its elements. If each row in the resulting matrix has an element equal to 0 where the column (j-index) of that

element is not reused in the 0-valued elements of any other row, then an optimal assignment can be derived.

If this is not the case, we find the number of horizontal and vertical lines needed to cover each 0 element (a *covered value*) in the matrix. If less than n lines were used, where n is the number of rows and columns in the cost matrix, then we subtract the minimum positive-valued row value from each row element which is *uncovered* (i.e. non-zero) and add the value to elements which are covered. We then reassess whether n lines are needed to cover the 0 values and repeat until this condition is met. The resulting matrix elements with values equal to 0 correspond to the optimal one-to-one pairing of elements in the original set with elements in the reconstructed set. This approach to the Hungarian method has a cubic time complexity, resulting in a substantial improvement over the naive method.

Despite the Hungarian method having the advantage of preserving a one-to-one correspondence between original and reconstructed set elements, Chamfer loss was deemed preferable for this project as:

- Reconstructed freeze-frames are used as a training signal to produce encoder-generated embeddings which capture meaningful spatial context information. The decoder-generated freeze-frames are not needed for any downstream tasks (e.g. synthetically generating realistic football player event freeze-frames).
- Players in freeze-frames are not uniquely identifiable, so enforcing one-to-one correspondence does not necessarily provide additional semantic information.
- Chamfer loss operates with $O(n^2)$, whereas Hungarian matching is $O(n^3)$. For a training set size $n \in [10^5, 10^6]$, such as in this project, and given that the constant factor of Chamfer distance is comparatively computationally cheaper than that of the Hungarian method, the reduction of time complexity by a factor of n makes using Chamfer loss more economical.

Future work with developing spatial embeddings from freeze-frames may wish to empirically contrast alternative reconstruction loss functions whilst considering the trade-off in terms of computation time.

The second component of this set transformer's loss function relates to the teammate, actor and goalkeeper flags which each player in each freeze-frame as, as seen in Figure 2. Given that the teammate, actor and goalkeeper statuses of each player are binary categorical variables, we can model their realisations as being sampled from Bernoulli distributions. The decoder can therefore predict a probability of the flag being equal to 1 and we can use this prediction to evaluate performance.

$$\mathcal{L}_{\text{BCE}} = \frac{1}{N}\sum_{i=1}^{N}(-y_i \log \hat{p}_i - (1-y_i)\log(1-\hat{p}_i)) \tag{10}$$

Binary cross entropy (BCE) loss provides a measure of how close predicted probabilties are to the true binary values. When the true value is 1, the second term evaluates to 0 and thus the loss is determined by the first term $-y_i \log \hat{p}_i$, which evaluates to 0 when the predicted probability is 1, and approaches ∞ when the limit of the predicted probability tends to 0. When the true value is 0, the first term cancels and the second term similarly provides a measure of loss which is 0 when the predicted probability is 0, and approaches ∞ when the loss approaches 1. This loss is calculated for each data instance individually and its average is taken to provide an overall measure of BCE loss.

To combine these two measures into a single loss function for the set transformer, we can use a weighted sum, where the weights λ_{xy} and λ_f are tunable hyperparameters. Equation 11 therefore constitutes the loss function used for this present project's set transformer.

$$\mathcal{L} = \lambda_{xy}\mathcal{L}_{\text{Chamfer}} + \lambda_f\mathcal{L}_{\text{BCE}} \tag{11}$$

4.5. Set Transformer Hyperparameter Tuning

Rather than training a single set transformer with arbitrary hyperparameters, it would be preferable to use an algorithm to choose values for the model's hyperparameters as to minimise its loss function, whilst avoiding overfitting. This process is known as hyperparameter optimisation or tuning.

One standard approach to hyperparameter tuning is grid search. With grid search, a discrete set of candidate values are provided for each model hyperparameter and each possible combination of these candidates is trialled. Grid search, whilst exhaustive of the search space, is an inefficient means of hyperparameter tuning, relative to alternative methods. With h hyperparameters in the model, and v candidate values across all hyperparameters, the search space with grid search is equivalent to v^h, which constitutes an exponential time complexity $O(v^h)$. Grid search's efficacy is further compounded by the fact that much time is spent exploring hyperparameter value combinations which yield poor performance, relative to the optimal combination available. Moreover, the search space provided must be discrete, which potentially misses more optimal hyperparameter values that may be found in a continuous search space. Whilst potentially useful for small models where there exists well-established prior expectations for appropriate hyperparameter values, alternative methods may be preferred.

An alternative approach can be found in random search. Random search draws n random samples of hyperparameter value combinations from the set of values provided and trials them. Rather than an exhaustive search of a discrete set of hyperparameter value combinations, random search enables sampling over both discrete sets of values and also continuous sampling distributions for hyperparameter values. Compared to grid search, this helps to reduce the time complexity of the search space to linear time, with search time simply being a linear function of the number of desired trials - $O(n)$ - whilst also allowing for more optimal values to be chosen via the use of hyperparameter value sampling distributions. Bergstra & Bengio (2012) argue that since hyperparameters for a given model often do not contribute evenly to the performance of the model (e.g. the learning rate may be more important that the dropout rate), grid search wastes time exploring hyperparameter value combinations which will not yield significant gains in performance. Instead, random search randomly samples values for all hyperparameters at each trial, meaning that the most important hyperparameters are always being tested with new unique values within a fixed computational budget. Grid search instead wastes computational budget on varying less important hyperparameters whilst keeping important hyperparameters constant, since it must exhaust all possible combinations. Random search therefore can explore the search space of important hyperparameters more efficiently, providing opportunity for random search to outperform grid search.

Despite this advantage over grid search, a limitation common to both grid search and random search is the fact that neither approach uses information gained from previous trials to improve the estimations made by future trials. Sampling efficiency could be drastically improved if an algorithm were to consider previous trials in its sampling of new candidate values.

One family of optimiser approaches which we can use to enhance hyperparameter sampling efficiency are Bayesian optimisers. In general, a Bayesian optimiser constructs a surrogate model to predict model performance y given a set of hyperparameter values x. This surrogate model

$p(y \mid x)$ produces its estimates based on some prior history of trials. After each trial, the predictions from this surrogate model are used to inform an acquisition function, which explores the search space to find regions which balance currently acquired knowledge on which hyperparameters lead to improved performance over less explored regions where uncertainty is higher and thus potential for further optimisation may exist. This acquisition function then samples hyperparameter values based on this tradeoff, which in turn updates the surrogate model and thus the process continues for some n number of trials. A Bayesian optimisier can therefore incorporate prior evidence acquired from previous trials to inform decisions on which hyperparameter values ought to be trialled next, enabling convergence towards a local optimium.

The present project will use a type of Bayesian optimiser known as a Tree-Structured Parzen Estimator (TPE), which is outlined in the next subsection (Section 4.5.1). This TPE is implemented via the Optuna Python library (Akiba et al., 2019). Table 1 provides a brief outline of the hyperparameters to be used for the set transformer in this project.

Hyperparameter	Description
Batch Size	The number of data instances processed before model parameters are updated.
Dropout	A random proportion of network activations set to zero during training.
Embedding Dimensions	The number of dimensions in the output layer of the set transformer encoder.
Flag Weight	A weighting applied to the Chamfer distance component of the set transformer loss function.
Hidden Dimensions	The number of dimensions of each hidden layer.
Learning Rate	A weighting applied to the derivative of the loss function with respect to model parameters to control the magnitude by which parameters are updated during training.
Number of Heads	The number of self-attention mechanisms which operate in parallel in each hidden layer.
Number of Hidden Layers	The number of hidden layers used within the set transformer encoder.
Weight Decay	An L2 regularisation applied to model weights.
XY Weight	A weighting applied to the binary cross-entropy component the set transformer loss function.

Table 1: Set Transformer Hyperparameter Descriptions

4.5.1. *Tree-Structured Parzen Estimatior (TPE)*

A TPE is a type of Bayesian optimiser which deviates from the general approach in a few key ways. Firstly, a TPE models the probability distribution of hyperparameter values, given model performance $p(x \mid y)$ - an inversion of the standard Bayesian optimiser approach. Thus, a TPE seeks to update the probability distributions of each hyperparameter in line with the prior evidence accumulated about which values lead to improved performance. $p(x \mid y)$ is defined by two derived densities: $l(x)$ and $g(x)$. Given a set of trials with candidate hyperparameters x and

observed loss values y, a parameter γ is defined so as to dervied the γth quantile of y referred to as y^* such that $p(y < y^*) = \gamma$. The exact value of γ may be a fixed value (i.e. always use $\gamma = 0.15$), or as in the case with Optuna's default implementation, an adaptive gamma function:

$$\gamma(n) = \min(\lceil 0.1n \rceil, 25) \tag{12}$$

This threshold y^* is used to define two probability density estimates for the set of hyperparameters. $l(x)$ constitutes probabilities density estimates for hyperparameter values where the observed loss is less than y^*, and $g(x)$ constitutes the probability density estimates for hyperparameters values where the observed loss is greater than or equal to y^*. As such, using the definition provided by Bergstra et al. (2011) on p.4:

$$p(x \mid y) = \begin{cases} l(x) & \text{if } y < y^* \\ g(x) & \text{if } y \geq y^* \end{cases} \tag{13}$$

To then choose an optimal set of hyperparameters for the next trial, TPE aims to sample hyperparameter values which maximise Expected Improvement (EI). EI therefore constitutes the acquisition function of the TPE algorithm. EI is given as the expectation of the improvement given the surrogate model's distribution, which we can define using Bayes' rule:

$$EI(x) = \int_{-\infty}^{y^*} (y^* - y) p(y \mid x)\, dy = \int_{-\infty}^{y^*} (y^* - y) \frac{p(x \mid y) p(y)}{p(x)}\, dy \tag{14}$$

Rather than computing this expectation directly, Bergstra, Bardenet, Bengio, & Kégl (2011) instead point out since any hyperparameter combination must come from either $l(x)$ or $g(x)$, we can express $p(x)$ as:

$$p(x) = \gamma l(x) + (1 - \gamma) g(x) \tag{15}$$

We can see from Equation 14 that $EI(x) \propto \frac{1}{p(x)}$. Therefore, by substitution and then rearrangement:

$$EI(x) \propto \frac{1}{\gamma l(x) + (1 - \gamma) g(x)} = \left(\gamma + (1 - \gamma) \frac{g(x)}{l(x)} \right)^{-1} \tag{16}$$

We can therefore see that $EI(x)$ is maximised when $\frac{g(x)}{l(x)}$ is smallest, or conversely when $\frac{l(x)}{g(x)}$ is maximised. This latter ratio is used by the acquisition function. The function samples m sets of hyperparameter values from $l(x)$ and computes $\frac{l(x)}{g(x)}$. The sample with the largest value for $\frac{l(x)}{g(x)}$ is then chosen as the next candidate hyperparameter value set. This then updates the surrogate model and the process continues for n trials. In order to provide densities $l(x)$ and $g(x)$ to the acquisition function, an initial startup period of trials is required. By default, Optuna's TPE sampler uses 10 startup trials before performing the TPE algorithm for some n number of trials.

To actually estimate $l(x)$ and $g(x)$, TPE uses kernel density estimation (KDE - a.k.a. the Parzen-Rossenblatt window method) (Parzen, 1962; Rosenblatt, 1956). KDE is a non-parametric method of estimating a probability density function given some set of n observations $\{x_1, ..., x_n\}$. In the case of TPE, these observations are the candidate hyperparameters from previous trials.

4.5.2. *Hyperband Pruning*

Each set of candidate hyperparameter values is trained across some number of epochs as specified during training. However, it is possible to evaluate trials more efficiently than to simply let each of then run their full course of epochs. In the context of this project, the number of epochs per trial is the *resource budget*. Hyperband pruning (Li et al., 2018) is a means to efficiently allocate this resource budget across trials as to avoid wasting training epochs on trials which are clearly performing poorly.

Firstly, hyperband tuning involves three key parameters: R_{min}, R_{max} and η. R_{min} is the minimum resource to be used per trial. As mentioned, for this project this is the minimum number of epochs each trial will train with. R_{max} is the resource budget we are choosing to employ (i.e. the maximum number of epoch per trial). η is known as the reduction factor.

Hyperband tuning uses the the successive halving algorithm to assess the performance of a trial during its progress. Whilst the standard successive halving algorithm involves running trials in parallel, the implementation in Optuna involves running trial in succession and thus an alternative method is employed. For each trial, we evaluate performance after the first rung is completed (i.e. R_{min}) and compare it to the performance of previous trials at that same rung. If performance is competitive with previous trials, we continue to the next rung, which is the current rung multiplied by the reduction factor η. We continue until either performance fails to compete, in which case the trial is terminated (i.e. *pruned*), or we reach the resource budget R_{max}.

What separates hyperband tuning from standard successive halving is that different sets of rungs are employed for some trials (known as brackets). Some brackets may involve starting with a smaller amount of resource, corresponding to more exploratory behaviour, whilst others may adopt a greater amount of resource initially, corresponding to a more exploitative approach. Trials are only compared to previous trials of the same bracket. This approach allows hyperband tuning to benefit from a balance of both exploratory or exploitative approaches, whereas standard successive halving must commit to one or the other. The number of brackets s_{max} used during tuning is determined as follows:

$$s_{\text{max}} = \left\lfloor \log_{\eta}\left(\frac{R_{\text{max}}}{R_{\text{min}}}\right) \right\rfloor \tag{17}$$

4.6. Classifier Training

Decroos, Bransen, Van Haaren, & Davis (2019) use 4 different supervised classification techniques to estimate scoring and conceding probabilities, enabling VAEP calculations. This project will replicate the implementation of these same 4 techniques as closely as possible, albeit they do not provide comprehensive details on the Python version used, librarie versions, or all model hyperparameters.

Given that VAEP requires two concurrent classifiers to be trained for scoring and conceding probability estimates, respectively, and the introduction of manually engineered spatial features and spatial embeddings will result in a total of 32 individual classifier models being trained, enabling the construction of: baseline VAEP; baseline + manual features; baseline + embeddings; and baseline + manual features + embeddings models. This ablation design allows for the impact of adding spatial context on model performance and behaviour to be investigated.

4.6.1. *Logistic Regression*

Logistic regression (Cox, 1958) is a classification technique which estimates the probability that a given data instance belongs to some reference class (i.e. an action results in a goal scored/ conceded vs no goal scored/conceded, within k events). Logistic regression learns a vector of weights $\mathbf{w}$ and bias b which minimises cross-entropy loss and produces a probability estimate for the data instance beloning to the reference class:

$$P(Y = 1 \mid X) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \tag{18}$$

Decroos, Bransen, Van Haaren, & Davis (2019) report the use of L2 Regularisation and a Limited-memory BFGS, which is replicated in this project.

L2 regularisation introduces an additional penalty term to the loss function (in this case cross-entropy loss, see Section 4.6.4). This penality term is some multiple $\lambda \in \mathbb{R}^+$ of the sum of squared weights:

$$\text{L2} = \lambda \sum_{i=1}^{N} w_i^2 \tag{19}$$

`scikit-learn` uses a hyperparameter $C = \frac{1}{\lambda}$ to control regularisation strength, which is set to 1 by default. Since the authors do not specify otherwise, this default will be maintained for training.

Limited-memory BFGS (Liu & Nocedal, 1989) is a computationally-efficient optimisation method, used in logistic regression to iteratively minimise the loss function during training. Again, this hyperparameter is the default for `scikit-learn`.

4.6.2. *Random Forest*

Classification tasks may also be performed by decision trees. We can treat a classification task as a form of recursive partitioning of the dataset until we obtain maximally *pure* partitions primarily consisting of data instances from a single classification class. Choosing a feature and subsequent splitting threshold which minimises impurity (see Section 4.6.5) until a maximum tree depth or minimum leaf node size is reached builds a decision tree which can be used to make classifications by applying a sequence of logical filters.

A random forest (Breiman, 2001) is an ensemble method which aggregates results from multiple decision trees, each trained on a random subset of the training set. In a classification task, this aggregation is employed by majority voting. For each data instance, the class predictions made by each tree in the forest are counted up and the class with the most "votes" is outputted as the predicted class for that instance. By having class predictions be made via majority voting from multiple decision trees, a random forest benefits from being less prone to overfitting and more robust to outliers in the dataset.

Decroos, Bransen, Van Haaren, & Davis (2019) indicate that they used 100 trees and 40 parallel threads to train their random forest models. Thus this project also used 100 trees. However, the number of parallel threads simply determines the number of concurrent processes used, which does not impact model behaviour other than via its runtime. Since my hardware is different to the authors', this hyperparameter will be chosen during training.

4.6.3. *Gradient Boosting*

Gradient boosting (Friedman, 2001) is another ensemble method involving decision trees. Gradient boosting methods construct a "strong" predictor by sequentially adding "weak" learners, where each new learner is trained to correct the errors of the previous ensemble.

$$\hat{y}_i = \sum_{t=1}^{T} \eta f_t(x_i), \quad f_t \in \mathcal{F} \tag{20}$$

Each function f_t is learned sequentially to minimise a loss function (like cross-entropy loss). At each iteration, the new weak learner is fit to the negative gradient of the loss with respect to the current model predictions. The learning rate η is used to determine how much the next tree contributes to model estimates.

4.6.3.1. *XGBoost*

XGBoost (Chen & Guestrin, 2016) builds upon standard gradient boosting by incorporating regularisation and introducing improved computational efficiency, even being capable of handling sparse data (i.e. data instances which are empty in some subset of features). Decroos, Bransen, Van Haaren, & Davis (2019) use 100 trees which are allowed to reach a maximum depth of 3 nodes. The learning rate η is equal to 0.1. Again, 40 parallel threads are used. Each of these hyperparameters (except the parallel threads) will be held constant in this project.

4.6.3.2. *CatBoost*

CatBoost (Prokhorenkova et al., 2019) is a gradient boosting method which is optimised for processing categorical predictors without requiring manual one-hot encoding. Decroos, Bransen, Van Haaren, & Davis (2019) retain the default hyperparameters for CatBoost (CatBoost Developers, n.d.), and again use 40 parallel threads.

4.6.4. *Loss Function: Cross-Entropy Loss*

Logistic regression, XGBoost and CatBoost each minimise cross-entropy loss in their classification predictions, according to their default implementations in the aforementioned Python libraries. Cross-entropy measures how "surprised" the model is by observations drawn from the true data-generating distribution $p(x)$, when evaluated under the predicted distribution $q(x)$. If the model tends to assign low probability to data instances which are probable under $p(x)$, then cross-entropy will be high. A low cross-entropy is found when $q(x)$ approximates $p(x)$ well.

$$H(p, q) = -\sum_{x \in X} p(x) \log q(x) \tag{21}$$

4.6.5. *Split Criterion: Gini impurity*

Whilst the split criterion used for the random forest models is not specified, the authors do indicate that they "used the implementation from the `scikit-learn` Python package" (Decroos, Bransen, Van Haaren, & Davis, 2019, p. 1860). It is thus inferred that they use the default split criterion of Gini impurity (Pedregosa et al., 2011; scikit-learn Developers, n.d.).

Gini impurity indicates the probability of assigning an incorrect label to a randomly chosen data instance by simply using the relative frequence of each label's occurrence in the data.

$$G_m = 1 - \sum_{k=1}^{K} \left(\frac{n_{m_k}}{n_m} \right)^2 \tag{22}$$

Decision trees within random forest classifiers, at each splitting decision, choose the feature and value threshold s which minimises the sum of the weighted Gini impurities of each partition m:

$$G_{m_{<s}/m_{>s}} = \frac{n_{m_{<s}}}{n_m} G_{m_{<s}} + \frac{n_{m_{>s}}}{n_m} G_{m_{>s}} \tag{23}$$

This results in the *purest* possible split and thus corresponds to the most discriminative splitting decision available. This process then iterates on each node in the decision tree until leaf nodes are reached which each only contain a single class (in this case, 0 or 1). Data instances can then be classified according to this derived decision tree.

4.7. Model Evaluation Metrics

4.7.1. *Area Under the Receiver Operating Characteristic Curve (AUC-ROC)*

A receiver operating characteristic curve (ROC) is produced by plotting the true-positive rate (TPR) against the false positive rate (FPR) for each probability threshold a model could use to classify a data instance into the reference class (i.e. the action results in a goal scored/conceded within k events). The result is an estimate of the probability that, given a random pair of instances where one is labelled 1 and the other labelled 0, the instance labelled 1 is assigned by the classifier a higher probability of belonging to the 1 class than the instance labelled 0. A higher AUC score therefore indicates strong performance by the classifier.

$$\text{AUC} = \frac{1}{mn} \sum_{j=1}^{n} \sum_{i=1}^{m} \phi(x_i, y_i) \quad \text{where } \phi(x_i, y_j) = \begin{cases} 1 & x_i > y_j \\ 0.5 x_i = y_j \\ 0 & x_i < y_j \end{cases} \tag{24}$$

4.7.2. *Brier Score*

Decroos, Bransen, Van Haaren, & Davis (2019) use the Brier score (Brier, 1950) to measure the performance of the VAEP classifier models by comparing the model's predicted probability of a data instance belonging to a class with its true class membership.

$$\text{BS} = \frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} (p_{ik} - y_{ik})^2 \tag{25}$$

This is effectively equivalent to the mean squared error, where predictions equate to probabilities of belonging to the kth class and the true values are a one-hot encoding of that data instance's membership of that k^{th} class. For binary classification this simplifies to:

$$\text{BS}_{\text{binary}} = \frac{1}{N} \sum_{i=1}^{N} (p_i - y_i)^2 \tag{26}$$

A lower Brier score indicates that the model's predicted probabilities are, on average, closer to the true outcomes.

4.7.3. *Cross-Entropy Loss*

Cross-entropy loss was introduced as a loss function in Section 4.6.4, but it may also be used as a classification model evaluation metric. This project intends to use cross-entropy loss to evaluate each of the two classifier models which constitute each VAEP model.

4.7.4. *Expected Calibration Error (ECE)*

Expected calibration error (ECE) (Naeini et al., 2015) is a performance metric used to evaluate the ability of a classification model to make accurate predictions. ECE creates m bins between 0 and 1 and assigns predictions to bins based on their probability estimates. For each bin, we take the accuracy of predictions in that bin o_i and subtract from it the average of the probability estimates in that bin e_i. This is weighted by the proportion of predictions in that bin. Each bin's score is the summed to provide the ECE of the model's predictions. A lower ECE is more desirable, indicating that model predictions are close to true frequencies.

$$\text{ECE} = \sum_{i=1}^{K} P(i) \cdot |o_i - e_i|$$

4.8. AUC Difference Significance Testing

To establish whether the observed differences in AUC scores are actually *surprising*, under the assumption that there are no significant differences in AUC scores between classifiers, we can employ the DeLong test (DeLong et al., 1988) on each pairwise comparison of features sets within the same classifier type and technique.

The DeLong test is a non-parametric significance test used to compare the AUCs of two ROC curves which are correlated, due to being evaluated on the same set of test labels. The test estimates the covariance matrix S between the two ROC curves being compared.

$$S = \begin{bmatrix} \text{Var}\left(\widehat{AUC}_1\right) & \text{Cov}\left(\widehat{\text{AUC}}_1, \widehat{\text{AUC}}_2\right) \\ \text{Cov}\left(\widehat{\text{AUC}}_1, \widehat{\text{AUC}}_2\right) & \text{Var}\left(\widehat{AUC}_2\right) \end{bmatrix} \tag{27}$$

S can then be used to derive the variance on the difference between the two AUC scores.

$$\text{Var}\left(\widehat{\text{AUC}}_1 - \widehat{\text{AUC}}_2\right) = \text{Var}\left(\widehat{\text{AUC}}_1\right) + \text{Var}\left(\widehat{\text{AUC}}_2\right) - 2\Big(\text{Cov}\left(\widehat{\text{AUC}}_1, \widehat{\text{AUC}}_2\right)\Big) \tag{28}$$

Taking the observed difference $\Delta = \widehat{\text{AUC}}_1 - \widehat{\text{AUC}}_2$ and dividing by the standard error of this difference provides a z statistic for *Delta*

$$z = \frac{\Delta}{\sqrt{\text{Var}\ (\Delta)}}$$

Under the null hypothesis that the true population AUC scores are equal $H_0 : \text{AUC}_1 = \text{AUC}_2$, we can calculate the p-value of this observed z statistic. Provided that p is less than or equal to a pre-specified significance threshold α, we can conclude that our estimate provides sufficient evidence to reject H_0. These significance tests will employ a standard significance threshold of 0.05.

An important caveat here is that given we are employing 6 pairwise comparisons within each classifier type and technique, the family-wise error rate of these 6 comparisons will exceed α. This is because each indiviual significance test has a 5% chance of producing a false positive (concluding a significant difference when there actually isn't one). The probability of obtaining at least 1 false positive within a family of m significance tests is:

$$\text{FWER} = 1 - (1 - \alpha)^m \tag{29}$$

This equates to a probability of obtaining a single false positive within a single family of pairwise AUC comparisons. The Holm-Bonferroni method (Holm, 1979) can be used to provide corrected p-values in light of their FWER.

This method first sorts each computed p-value for the m significance tests in ascending order and adjusts the significance threshold using the ratio $\frac{\alpha}{m+1-k}$, where k is the index (starting at 1) of each p-value in the sorted list. The first p-value is therefore testing if $p \leq \frac{\alpha}{m+1-1}$. If this is true, we move to the next p-value in the list and increment k. This continues until either the condition is not met, in which case the remaining significance tests are deemed non-significant, or all m tests have been corrected.

The implementation of signficance threshold adjustment in the `statsmodels` package involves adjusting the computed p-values themselves, rather than adjusting the significance threshold (Perktold et al., 2024; Wright, 1992, pp. 1009). Instead, we can take each p-value computed from the DeLong tests, again sorted in ascending order, and apply the following adjustment to each:

$$\tilde{p}_{(i)} = \max_{1 \leq j \leq i} \left\{ (m - j + 1) p_j \right\} \tag{30}$$

4.9. Model Interpretability

4.9.1. *Permutation Importance*

Permutation importance is a method for estimating feature importance by randomly permuting one or more features and evaluating the change in model performance metrics. Features which, when permuted, result in a larger drop in model performance can be inferred to be more important in model estimations. Care should be taken with multicolinearity, as permutations will not entirely disconnect their relationship with others model features since those relationships can continue to exist via their colinear counterparts. Permutation importance will thus be used in this project to highlight which features or set of features are most determinative of the VAEP values assigned to actions.

4.9.2. *Partial Dependence Plots (PDPs)*

Partial dependence plots (PDPs) (Friedman, 2001) are used to visualise the relationship between some set of features (usually just 1 or 2) and the predictions made by a machine learning model, holding other features constant. In this case, we can use PDP plots to explore our interpretable feature set, that is the baseline and manually-engineered features, as opposed to the spatial embeddings whose dimensions are not directly interpretable.

4.9.3. *k-Means Clustering*

We can also explore the spatial embeddings via k-means clustering, similarly to how DeepCluster applies k-means to image embeddings generated by a convolutional neural network (CNN) (Caron et al., 2019). k-means is an unsupervised clustering algorithm which finds

optimal locations for some k centroids within the d-dimensional feature space. The algorithm initialises k centroids within the feature space, assigns data points to their nearest centroid and then recalculates the location of each centroid to the position which minimises the distance to its assigned data points. The data points are then reassigned and the algorithm iterates until convergence. A number of distance measures may be used, with euclidean distance being most common.

$$\text{dist } (x_i, x_n) = \sqrt{\sum_{d=1}^{D} \left(x_{i_d} - x_{n_d}\right)^2} \tag{31}$$

The benefit of using k-means clustering with the present project's spatial embeddings is that underlying relationships represented by these embeddings can be explored by comparing the properties of clusters. For instance, player position heatmaps for each cluster could be plotted to potentially reveal notable team shapes, or if certain clusters tend to involve certain action types over others.

Given that this clustering is being applied to embeddings, it would be preferable for the distance metric to reflect the notion of similarity within an embedding space. Embedding similarity can be determined by cosine similarity, whereby the cosine of the angle between the two vectors indicates how similar they are: a smaller angle corresponding to more similarity. For two embedding vectors $\vec{a}$ and $\vec{b}$, their cosine similarity is given by:

$$\cos(a, b) = \frac{\vec{a} \cdot \vec{b}}{\parallel \vec{a} \parallel_2 \times \parallel \vec{b} \parallel_2} \tag{32}$$

If we L2 normalise the embedding vectors such that $\hat{a} = \frac{a}{\parallel a \parallel_2}$ and $\hat{b} = \frac{b}{\parallel b \parallel_2}$, the squared Euclidean distance $\parallel \hat{a} - \hat{b} \parallel^2$ of the two normalised vectors is a monotonic transformation of their cosine similarity:

$$\begin{aligned}
\left\|\hat{a} - \hat{b}\right\|_2^2 &= \left(\hat{a} - \hat{b}\right) \cdot \left(\hat{a} - \hat{b}\right) \\
&= \hat{a} \cdot \hat{a} - 2\hat{a} \cdot \hat{b} + \hat{b} \cdot \hat{b} \\
&\because \hat{a} \cdot \hat{a} = \hat{b} \cdot \hat{b} = 1 \\
&\therefore \left\|\hat{a} - \hat{b}\right\|_2^2 = 1 - 2\hat{a} \cdot \hat{b} + 1 \\
&= 2 - 2\hat{a} \cdot \hat{b} \\
&\boxed{\therefore \left\|\hat{a} - \hat{b}\right\|_2^2 = 2\left(1 - \hat{a} \cdot \hat{b}\right)} \\
&\because \cos(a, b) = \frac{\vec{a} \cdot \vec{b}}{\left\|\vec{a}\right\|_2 \left\|\vec{b}\right\|_2}, \quad \hat{a} = \frac{\vec{a}}{\left\|\vec{a}\right\|_2}, \quad \hat{b} = \frac{\vec{b}}{\left\|\vec{b}\right\|_2} \\
&\boxed{\therefore \hat{a} \cdot \hat{b} = \frac{\vec{a} \cdot \vec{b}}{\left\|\vec{a}\right\|_2 \left\|\vec{b}\right\|_2} = \cos(a, b)} \\
&\boxed{\therefore \parallel \hat{a} - \hat{b} \parallel_2^2 = 2\left(1 - \hat{a} \cdot \hat{b}\right)}
\end{aligned} \tag{33}$$

As such, prior to performing k-means clustering with these embeddings, L2 normalisation will be applied to them.

4.9.3.1. *Choosing* k

By fitting a model to the data with varying values of k, we can use a number of metrics to evaluate how appropriate the chosen value of k is for clustering the given data points.

4.9.3.1.1. *Inertia*

The hyperparameter k can be chosen by considering the marginal reduction in *inertia* as k increases. A suitable k can be chosen at the point where this marginal reduction becomes negligible (this is known as the *elbow method*):

$$\text{Inertia} = \sum_{j=1}^{K} \sum_{x_i \in C_j} \parallel x_i - \mu_j \parallel^2 \tag{34}$$

Whilst commonly used in deriving k and a somewhat informative heuristic, the elbow method is inherently statistically arbitrary as a means for determining the appropriate number of clusters (Schubert, 2023).

4.9.3.1.2. *Average Silhouette Score*

An additional metric is the average *silhouette score* (Rousseeuw, 1987) for each value of k. For each data instance i, we can determine its average distance to others instances in the same cluster (known as *cohesion* $a(i)$) and its average distance to instances in the next nearest cluster (known as *separation* $b(i)$). Taking the differences between the separation and cohesion and dividing by the maximum of the two, we get the data instance's silhouette score.

If separation is lower than cohesion for a data instance, the silhouette score will be negative and thus suggests the data instance is closer to some other cluster(s) relative to its distance to its assigned cluster. Likewise, greater cohesion over separation suggests the data instance is appropriately assigned to its cluster, in terms of the distance metric used.

An average can then be taken across all data points and this average score can be compared across different values for k, where 1 indicates good overall clustering and -1 poor:

$$\text{SC} = \frac{1}{N} \sum_{i=1}^{N} \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \tag{35}$$

4.9.3.1.3. *Calinski-Harabasz Index (CHI)*

We may also appeal to the Calinski-Harabasz index (CHI) (Caliński & Harabasz, 1974) in assessing a suitable k. CHI is just a ratio of the between-cluster sum of squares (BCSS) and the within-cluster sum of squares (WCSS, i.e. the inertia), where the former is the sum of squared distances between cluster centroids and the latter is the sum of squared distances of each data point to its respective cluster. Each of these sum of squares measures are normalised by their respective degrees of freedom.

$$\text{CHI} = \frac{\frac{1}{k-1} \sum_{i=1}^{k} n_i \parallel c_i - c \parallel^2}{\frac{1}{n-k} \sum_{i=1}^{k} \sum_{x \in C_i} \parallel x - c_i \parallel^2} \tag{36}$$

4.9.3.1.4. *Davies-Bouldin Index (DBI)*

Lastly, the Davies-Bouldin index (DBI) provides a simultaneous evaluation of how separated each cluster is and how compact data instances are to their assigned cluster. For each pair of clusters i, j, we can consider how their individual compactness (S_i and S_j) and their relative distances compare by taking the ratio of the sum of their compactness to the distance of their centroids.

$$R_{ij} = \frac{S_i + S_j}{M_{ij}} \tag{37}$$

S_i and M_{ij} can, in principle, be any measure of internal compactness or cluster separation, respectively. `scikit-learn` implements the average Euclidean distance between data instances and their cluster's centroid for internal compactness, and Euclidean distance between cluster centroids for cluster separation.

A lower value for R_{ij} indicates the clusters i, j are well-separated and internally compact. As such, for each cluster i, we can consider its *worst-case scenario* by finding its corresponding j^{th} cluster which maximises R_{ij}. Taking the sum of each of these maximum R_{ij} values and dividing by the number of clusters k gives a measure of the average clustering performance $\bar{R}$, where minimising the measure optimises clustering performance:

$$\bar{R} = \frac{1}{k} \sum_{i=1}^{k} \max_{i \neq j} R_{ij} \tag{38}$$

4.9.4. *Uniform Manifold Approximation and Projection (UMAP)*

In order to visualise and explore the spatial embeddings, uniform manifold approximation and projection (UMAP) (McInnes et al., 2020) can be used as a dimensionality reduction technique to represent these embeddings in a 2 or 3 dimensional space. The color of data points within a UMAP can then be determined by properties such as the VAEP value, scoring/conceding probability or k-means clusters, allowing for the observation of emergent representational structure within the spatial embeddings, should any structure arise.

For each data point, UMAP locates its k nearest neighbours (where k is a pre-specified hyperparameter). A directed graph is constructed between each data point and its neighbours, where the weight of each directed connection is determined as follows:

$$w\big(x_i, x_{i_j}\big) = \exp\left(\frac{-\max\big(0, d\big(x_i, x_{i_j}\big) - \rho_i\big)}{\sigma_i}\right) \tag{39}$$

$d\big(x_i, x_{i_j}\big)$ is the raw distance between points i and j (e.g. Euclidean distance). ρ_i is the raw distance of the closest neighbour to point i. σ_i is a scaling parameter used to control how rapidly weights decrease as a function of distance. σ_i is uniquely determined for each data point i such that it satisties the following equivalence:

$$\sum_{j=1}^{k} \exp\left(\frac{-\max\big(0, d\big(x_i, x_{i_j}\big) - \rho_i\big)}{\sigma_i}\right) = \log_2(k) \tag{40}$$

Here, the hyperparameter k is the aforementioned number of nearest neighbours each data point is pre-specified to have. This scale parameter ensures the size of each neighbourhood is comparable between regions of varying densities.

With each directed weighted graph for each data point constructed, these graphs can then be combined into a global directed weighted graph for the entire data set. To ensure symmetry between connections from points i and j (which is not ensured when calculating $w\big(x_i, x_{i_j}\big)$), weights are updated as follows:

$$w_{ij} = w\big(x_i, x_{i_j}\big) + w\big(x_j, x_{j_i}\big) - w\big(x_i, x_{i_j}\big)w\big(x_j, x_{j_i}\big) \tag{41}$$

Each data point is then assigned an intialised lower-dimensional spectral embedding (e.g. 2 dimensions). To optimise these embeddings such that the derived lower-dimensional representation as closely resembles the original n-dimensional data set, a new global directed weighted graph of this lower-dimensional embeddings space is constructed and made to resemble the global directed weighted graph from the n-dimensional data set. The optimisation adjusts the locations of points in the lower-dimensional space such that they are positioned closer to points with which they share a stronger weighting in the original graph and further away from points with which they share a weaker weighting in the original graph. This optimisation is achieved via stochastic gradient descent, which minimises the following cross-entropy loss function:

$$\mathcal{L} = \sum_{(i,j)} w_{ij} \log\left(\frac{w_{ij}}{\hat{w}_{ij}}\right) + (1 - w_{ij}) \log\left(\frac{1 - w_{ij}}{1 - \hat{w}_{ij}}\right) \tag{42}$$

5. Results

5.1. Set Transformer Training

The set transformer used to generate spatial embeddings was trained on 652,041 atomic-spadl actions, with a validation set of 95,933 used for hyperparameter tuning. A final test set of 191,180 instances was used to test the best performing model following hyperparameter tuning. This best performing model was trained for 20 epochs, whereas models with candidate hyperparameter during tuning were trained for 10 epochs, unless pruned by the hyperband pruner.

A total of 50 trials were used for hyperparameter tuning. Table 2 shows the set of candidate values which the TPE sampler drew from. As indicated by the `Data Type` column, most hyperparameters were passed as categorical sets, whilst those which were continuous allowed the TPE sampler to sample from any value within the specified range, inclusive. The learning rate and weight decay were additionally specified to be sampled log-uniformly.

Hyperparameter	Candidate Values	Data Type
Batch Size	[256, 512, 1024]	Categorical
Dropout	[0.0, 0.1, 0.2, 0.3]	Categorical
Embedding Dimensions	[16, 32, 64]	Categorical
Flag Weight	(0.05, 0.5)	Continuous
Hidden Dimensions	[128, 256, 512]	Categorical

Hyperparameter	Candidate Values	Data Type
Learning Rate	(0.0001, 0.005)	Continuous (Logarithmic)
Number of Heads	[2, 4, 8]	Categorical
Number of Hidden Layers	[1, 2, 3, 4]	Categorical
Weight Decay	(1e-06, 0.001)	Continuous (Logarithmic)
XY Weight	(0.5, 2.0)	Continuous

Table 2: Hyperparameter Candidate Values

As shown in Table 3, trial 41 performed best in terms of minimising the weighted Chamfer + BCE loss function. The hyperparameters derived by the TPE sampler are of interest here for interpreting the embeddings which were subsequently generated by this trained set transformer.

trial	value	batch size	output dim	flag weight	hidden dim	drop -out	lr	num heads	num layers	weight decay	xy weight
0	nan	256	16	0.204	128	0.3	4.7e-04	2	4	1.5e-04	1.42
1	nan	256	64	0.242	512	0.2	4.0e-03	4	3	2.8e-05	1.98
2	47.822	512	64	0.444	256	0.0	1.2e-03	4	1	1.9e-06	1.83
3	33.558	512	16	0.299	256	0.0	4.0e-04	8	2	6.0e-05	1.54
4	88.493	512	64	0.431	256	0.2	3.5e-04	8	3	1.0e-06	1.98
5	nan	256	32	0.379	512	0.2	4.7e-03	4	2	6.1e-04	1.35
6	893.416	512	16	0.349	512	0.2	1.5e-03	8	4	1.0e-05	1.89
7	nan	1024	64	0.230	256	0.3	2.2e-04	2	3	4.2e-06	1.58
8	62.816	512	32	0.344	128	0.2	1.5e-04	2	1	1.7e-05	1.22
9	56.870	1024	32	0.259	128	0.2	2.6e-04	8	4	4.2e-05	1.44
10	17.773	512	16	0.054	256	0.0	1.0e-04	8	2	9.1e-04	0.76
11	22.031	512	16	0.051	256	0.0	7.2e-04	8	2	7.9e-04	0.76
12	15.992	512	16	0.053	256	0.0	1.0e-04	8	2	9.4e-04	0.71
13	nan	512	16	0.051	256	0.1	1.0e-04	8	2	2.6e-04	0.55
14	34.458	1024	16	0.132	256	0.0	1.1e-04	8	2	3.5e-04	0.94
15	20.950	512	16	0.135	256	0.0	1.7e-04	8	2	1.3e-04	0.95
16	nan	512	16	0.116	256	0.1	1.6e-04	8	2	9.6e-04	0.55
17	19.139	512	16	0.098	256	0.0	1.0e-04	8	2	3.6e-04	0.81
18	nan	256	16	0.176	512	0.0	2.4e-04	2	1	1.5e-04	1.12
19	28.115	1024	32	0.086	128	0.0	1.6e-04	4	2	5.1e-04	0.70
20	nan	512	16	0.165	256	0.3	2.7e-04	8	2	9.4e-04	0.98
21	16.221	512	16	0.082	256	0.0	1.2e-04	8	2	3.0e-04	0.76
22	14.509	512	16	0.075	256	0.0	1.4e-04	8	2	2.2e-04	0.67
23	11.658	512	16	0.096	256	0.0	1.9e-04	8	2	8.3e-05	0.65
24	nan	512	16	0.164	256	0.1	2.0e-04	8	2	7.4e-05	0.62
25	38.201	512	16	0.112	256	0.0	5.8e-04	8	2	9.0e-05	0.90
26	10.133	512	16	0.496	256	0.0	3.4e-04	8	1	1.9e-04	0.62
27	15.584	512	16	0.283	128	0.0	3.3e-04	8	1	1.8e-04	0.62
28	nan	256	32	0.491	512	0.0	8.3e-04	4	1	3.0e-05	0.53
29	nan	1024	64	0.183	256	0.3	4.4e-04	2	4	1.1e-04	1.14

trial	value	batch size	output dim	flag weight	hidden dim	drop -out	lr	num heads	num layers	weight decay	xy weight
30	nan	256	16	0.204	128	0.1	5.7e-04	2	1	2.0e-04	0.84
31	16.109	512	16	0.278	128	0.0	3.4e-04	8	1	1.9e-04	0.65
32	14.740	512	16	0.312	128	0.0	3.1e-04	8	1	5.3e-05	0.63
33	14.430	512	16	0.434	128	0.0	1.9e-04	8	1	5.7e-05	0.52
34	11.963	512	16	0.491	128	0.0	2.5e-03	8	1	1.8e-05	0.52
35	12.841	512	64	0.496	128	0.0	2.4e-03	4	1	1.4e-05	0.51
36	nan	512	64	0.499	128	0.3	2.7e-03	4	1	5.3e-06	0.58
37	400.440	256	64	0.476	128	0.0	2.6e-03	4	3	2.0e-05	0.84
38	14.461	512	64	0.403	128	0.0	1.6e-03	4	1	8.8e-06	0.51
39	nan	512	64	0.460	512	0.0	3.1e-03	4	1	2.4e-06	0.71
40	493.115	512	64	0.401	128	0.2	1.6e-03	4	3	1.2e-05	1.04
41	8.841	512	64	0.435	128	0.0	1.2e-03	8	1	3.6e-05	0.51
42	12.273	512	64	0.458	128	0.0	2.2e-03	8	1	3.1e-05	0.58
43	60.331	512	64	0.462	128	0.0	1.2e-03	8	1	3.1e-05	1.71
44	286.806	512	64	0.450	128	0.0	4.8e-03	8	4	2.0e-05	0.60
45	35.508	512	64	0.413	128	0.2	2.0e-03	8	1	3.9e-05	0.74
46	56.772	1024	32	0.362	512	0.0	4.0e-03	8	1	7.1e-06	0.59
47	nan	512	64	0.425	128	0.3	1.1e-03	8	1	2.5e-05	0.80
48	316.971	512	64	0.382	128	0.0	1.9e-03	8	3	4.2e-05	0.67
49	416.112	256	32	0.476	128	0.1	3.8e-03	8	4	9.0e-05	0.88

Table 3: Optuna Hyperparameter Tuning Results

The hyperparameters from this trial were used to train the final set transformer model instance, which achieved a loss of 9.19 on the held-out test set. Figure 3 shows the training and validation loss curves for this final model. This plot shows fast convergence with good generalisability, given that both loss curves drop quickly and remain close together across the number of epochs.

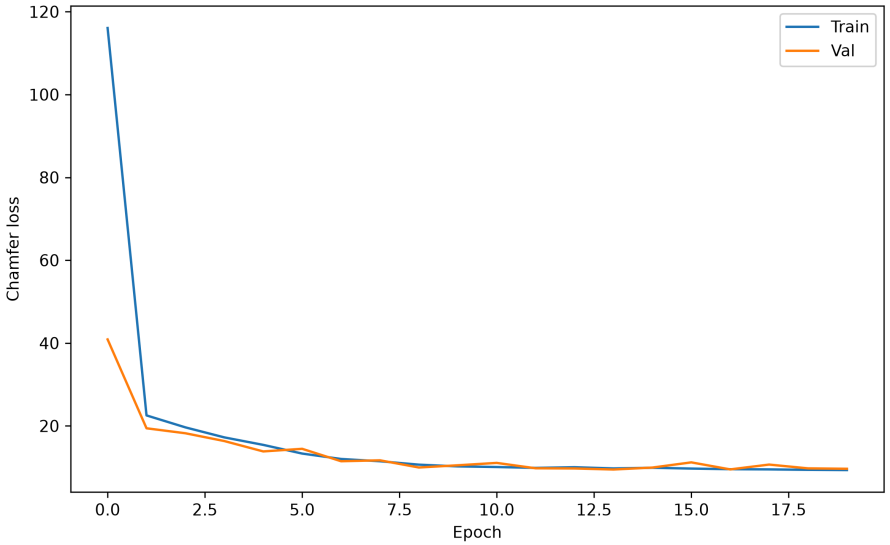


Figure 3: Set Transformer Training Loss Curve

An output dimension of 64 corresponds to the number of spatial embedding dimensions used during inference. The candidate output dimensions were intentionally set to always be smaller than the hidden dimension sizes. This forces a compression of information into a fixed-dimensional representation, with additional width in the output layer providing room for added complexity if appropriate, however this may also introduce the risk of losing generalisability if made too wide (Chen & Guo, 2023). Furthermore, given that these fixed-dimensional embedding representations serve as features in the downstream classification models, encouraging compression has the added benefit of contributing to dimensionality reduction and thus maintaining a parsimonious feature set.

Lee, Lee, Kim, Kosiorek, Choi, & Teh (2019) note that "Since the output of SAB [Set Attention Block] contains information about pairwise interactions among the elements in the input set X, we can stack multiple SABs to encode higher order interaction" (p.3). Here, an SAB is just a hidden layer of the set transformer encoder. As with other deep learning techniques then, additional hidden layers (SABs) generally correspond to the encoding of increased levels of abstract representation between set elements (players). Given that the TPE sampler appeared to find a value of 1 for the number of hidden layers to be optimal, it would potentially suggest that latent relationships between players within freeze-frames is relatively simple and thus does not require multiple layers of abstraction to produce embedding representation which minimise loss. This seems relatively intuitive, given that freeze-frames consist of 22 players at most.

That being said, one might suggest that this could be an artefact of the amount of training time afforded (10 epochs), whereby deeper representations require additional training time to update the additional layer parameters. However, Figure 4 shows that, after training 4 copies of the final model using 20 epochs on a subset of the train/val/test datasets and varying just the number of hidden layers, a hidden layer size of 1 still achieves optimal performance. Having 2 hidden layers appears to achieve near identical loss to having 1, but it appears that adding more hidden layers leads to loss increasing dramatically. We can also see that the hidden layer sizes of 3 and 4 appear to have near constant loss across the training epochs which is much higher than that seen in sizes 1 and 2, which also appear to show convergence across the epochs. This constant high loss for hidden layer sizes 3 and 4 would suggest that additional model depth is harmful for producing embeddings which minimise reconstruction loss. This provides evidence against the counterargument that deeper models simply might require more training time to converge and instead suggests a shallower encoder is more appropriate for minimising loss.

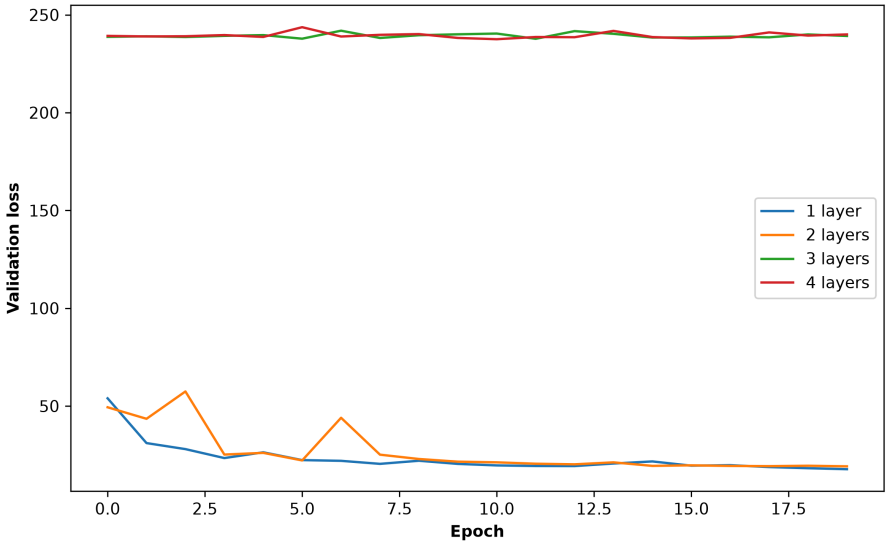


Figure 4: Comparison of Training Loss Curves for Each Value for Number of Hidden Layers [1, 2, 3, 4]

5.1.1. *K-means Clustering: Spatial Embeddings*

To explore the spatial representations which may be encoded in these spatial embeddings, we can use k-means clustering on the derived embeddings. Given that not all actions in the dataset have associated freeze-frames, those actions without freeze-frames are assigned embeddings where all dimensions equate to 0. Since such embeddings do not encode any meaningful information, they are excluded from this clustering analysis, which instead focuses upon embeddings which, in principle, have the possibility of encoding latent representations.

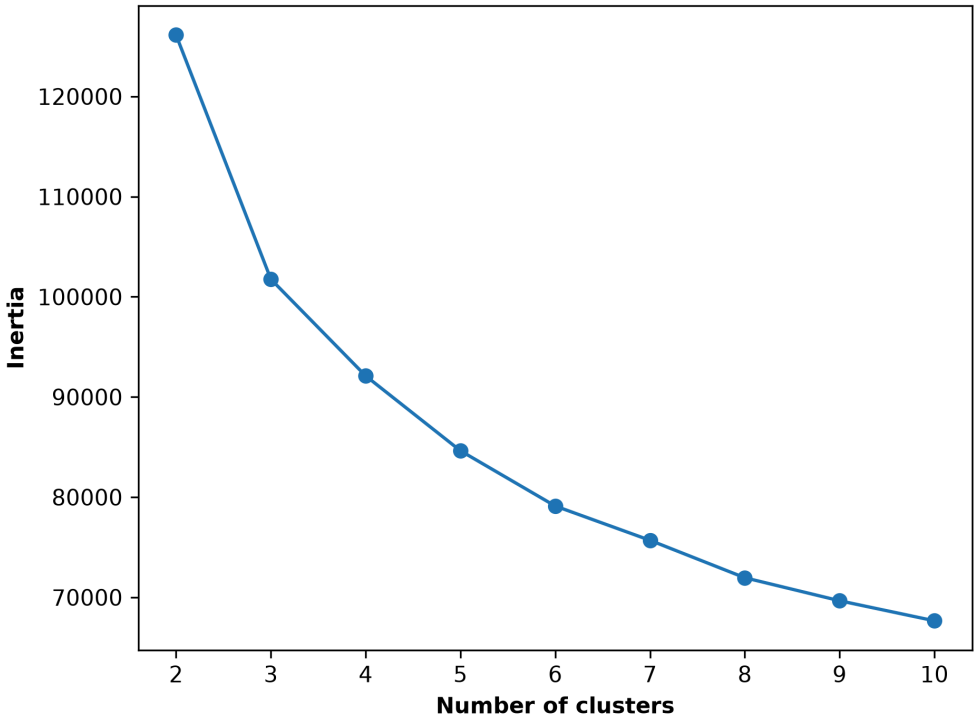


Figure 5: K-means Elbow Method Plot

Figure 5 provides the elbow method plot, indicating the marginal trade-off betweeen the number of clusters k and the inertia of each model. A naive evaluation of this plot would arguably favour $k = 3$ as this appears to constitute the "elbow" of the curve, although such a conclusion is inherently subjective. Instead, we can leverage additional model metrics to determine an appropriate value of k.

k	inertia	silhouette	calinski harabasz	davies bouldin
2	1.26e+05	0.38	5.84e+05	1.11
3	1.02e+05	0.25	4.55e+05	1.37
4	9.21e+04	0.24	3.62e+05	1.52
5	8.46e+04	0.19	3.13e+05	1.55
6	7.91e+04	0.18	2.79e+05	1.60
7	7.57e+04	0.16	2.49e+05	1.65
8	7.20e+04	0.15	2.30e+05	1.67
9	6.97e+04	0.14	2.11e+05	1.72
10	6.77e+04	0.13	1.96e+05	1.79

Table 4: K-means Hyperparameter Results

Table 4 instead provides stronger evidence for $k = 2$ being the appropriate number of clusters. We can see that $k = 2$ produces the highest average Silhouette score (0.37), indicating weak clustering, and the lowest Davies-Bouldin index (1.11). This additional evidence provides an indication of between-cluster separation and within-cluster compactness, whilst inertia alone

considers just compactness. As such, it is arguable that more emphasis be placed on the Silhouette score and Davies-Bouldin index.

Using $k = 2$, we can explore the properties of each derived cluster. Below are the heatmaps of each cluster, indicating the average x-y co-ordinates of freeze-frames beloning to each cluster, where the attacking direction is from left to right.

We can see from Figure 6 and Figure 7 that there is a notable difference in average co-ordinates of actions belonging to each cluster. cluster 0 appears to represent actions around the center of the pitch, whilst cluster 1 appears to represent actions occurring just outside and inside the opposition penalty area. Given that their average pitch locations are notably different, we might expect further properties of these clusters to vary also.

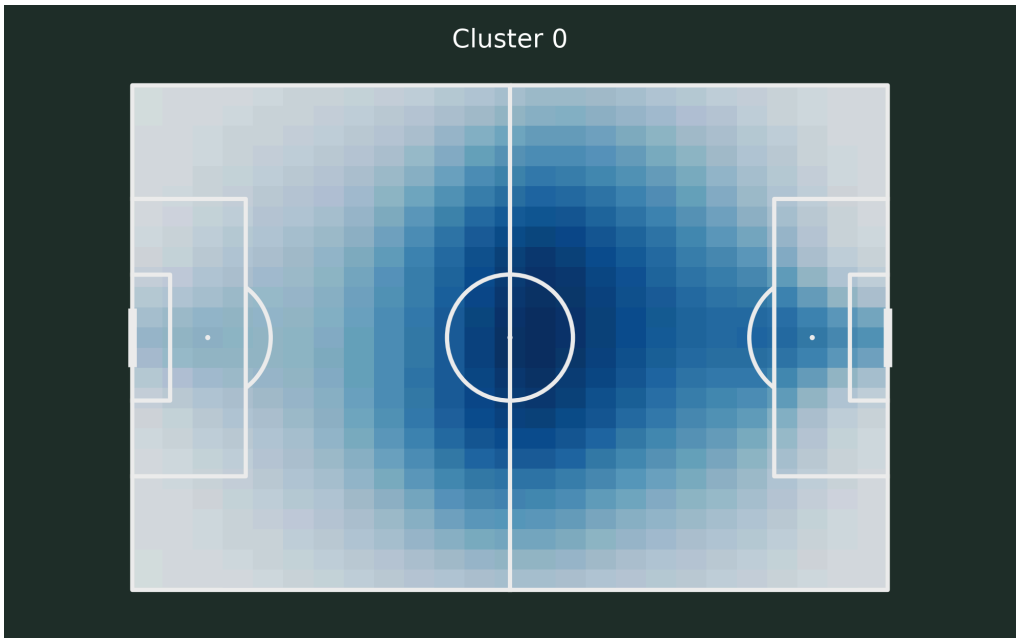


Figure 6: Pitch Heatmap of Cluster 0

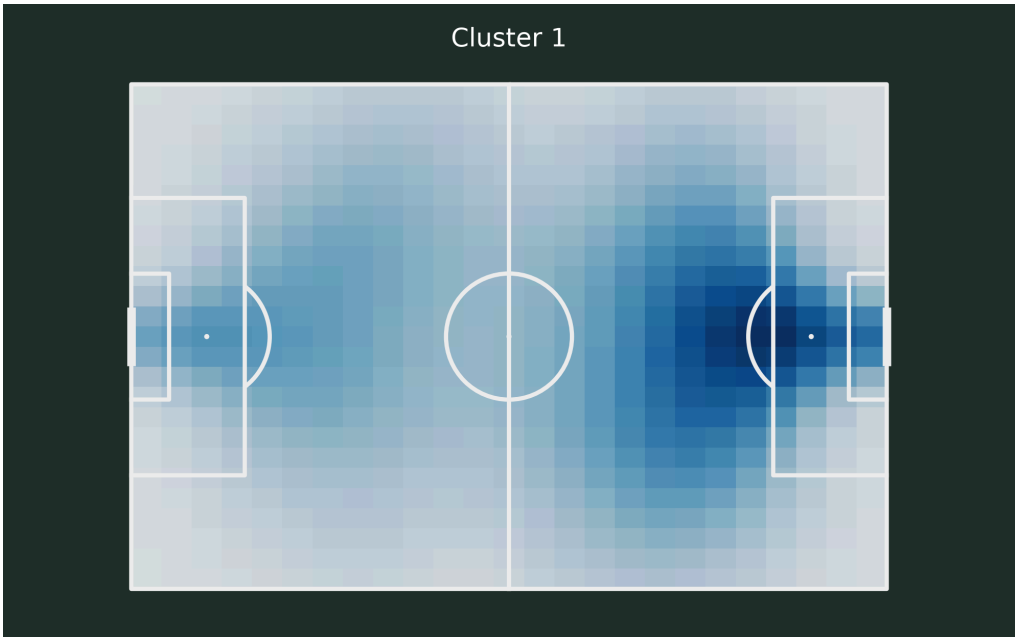


Figure 7: Pitch Heatmap of Cluster 1

We can also compare each cluster's distribution of VAEP values. Given that we have 2 clusters where each action has an associated VAEP value, we can use a two-sided t-test to examine if the average VAEP values of each of the two clusters significantly differ.

In order to use a two-sided t-test, we typically desire residuals to be normally distributed. Here, the residuals are the differences between each observed VAEP value and the average VAEP of the cluster that observation belongs to. Given that we have a very large sample of spatial embeddings ($n = 776,576$), the normality assumption can be relaxed as a t-test is robust to deviations from residual normality under large ($n \gg 500$) sample sizes (Lumley et al., 2002).

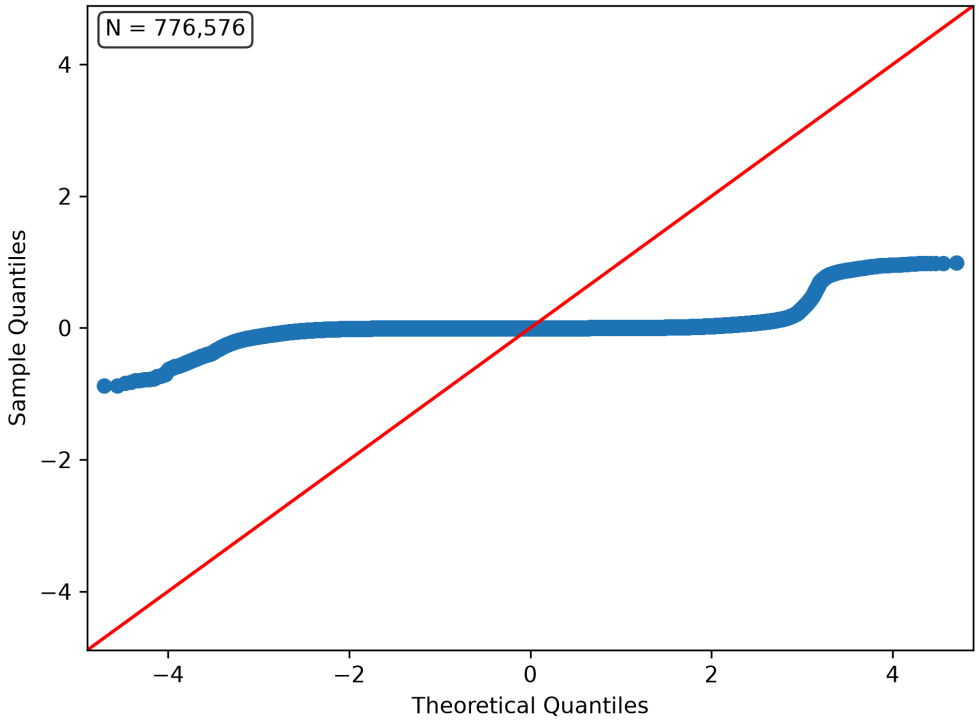


Figure 8: Residual QQ-Plot for Two-Sided Welch's t-test

Of greater importance is the assumption of homogeneity of variance, which Table 5 shows is not met by our sample, where cluster 0 and 1 show variances of 0.0008 and 0.0011, respectively. As such, a variant of the standard t-test known as Welch's t-test (Welch, 1951) can be used, which does not assume equal variance across comparison groups.

Count	Mean	Var
501617	0.002415	0.000804
274959	0.003074	0.001134

Table 5: Mean and Variance of VAEP Values for each K-Means Label

Results from the two-sided Welch's t-test reveal at a signficant difference between group means, $t(489849.35) = -8.71, p < .001,$ Cohen's $d = .022$. It should be noted that the Cohen's d effect size is very small, indicating that the diference between the means of each group is very small relative to the variation within each group.

Exploring how the average VAEP value changes across different quantiles of each cluster's distribution of VAEP values shows which parts of the distributions contribute to these averages for each cluster.

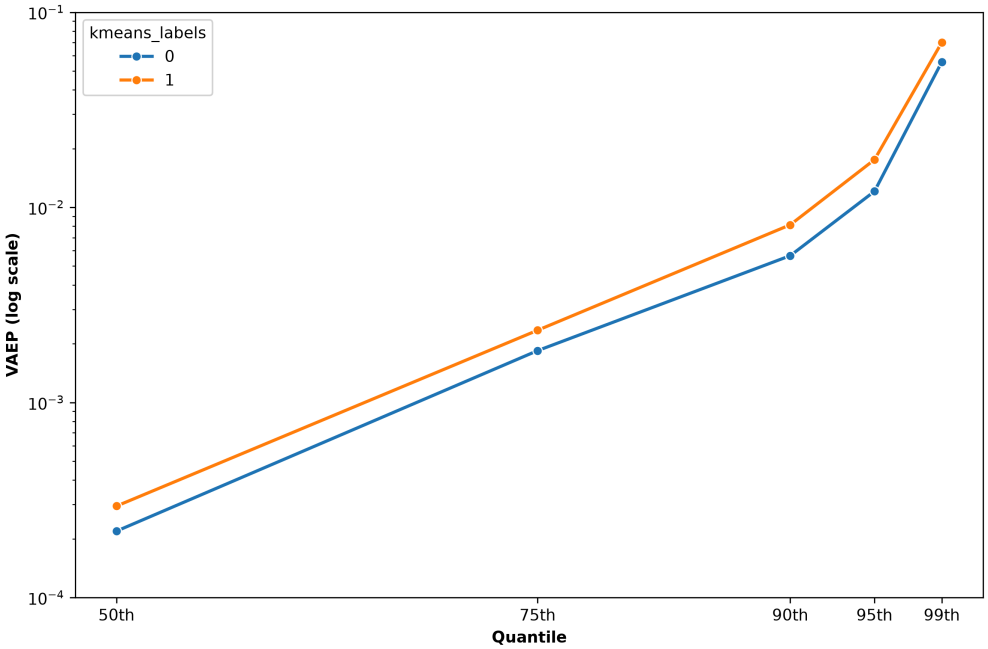


Figure 9: Comparison of Average VAEP at each Quantile for Each Spatial Embedding Cluster

We can see that cluster 1 shows higher average VAEP values than cluster 0 across all quantiles from the 50th to the 99th. Given that the overall effect size appears very small, the comparison of average VAEP values across the upper quantiles shows that this difference is somewhat larger for higher-valued actions.

5.1.2. *UMAP*

A UMAP can help to visualise the cluster structure within the spatial embeddings. Figure 10 shows that the two k-means clusters occupy broadly distinct regions of the 2-dimensional UMAP across the tested hyperparameter values, with cluster 0 generally positioned to the left of cluster 1. The broad separation between the clusters remains relatively consistent across different values of the number of neighbours and minimum distance, suggesting that this aspect of the embedding structure is not strongly dependent on the selected UMAP configuration. Within each k-means cluster, however, there also appears to be a secondary two-region structure. This internal separation becomes increasingly pronounced as the number of neighbours is increased, particularly at larger values of `n_neighbours`. Thus, while the primary distinction between the two k-means clusters remains stable, the UMAP suggests weak additional structure within each cluster that is influenced by the scale over which neighbourhood relationships are considered.

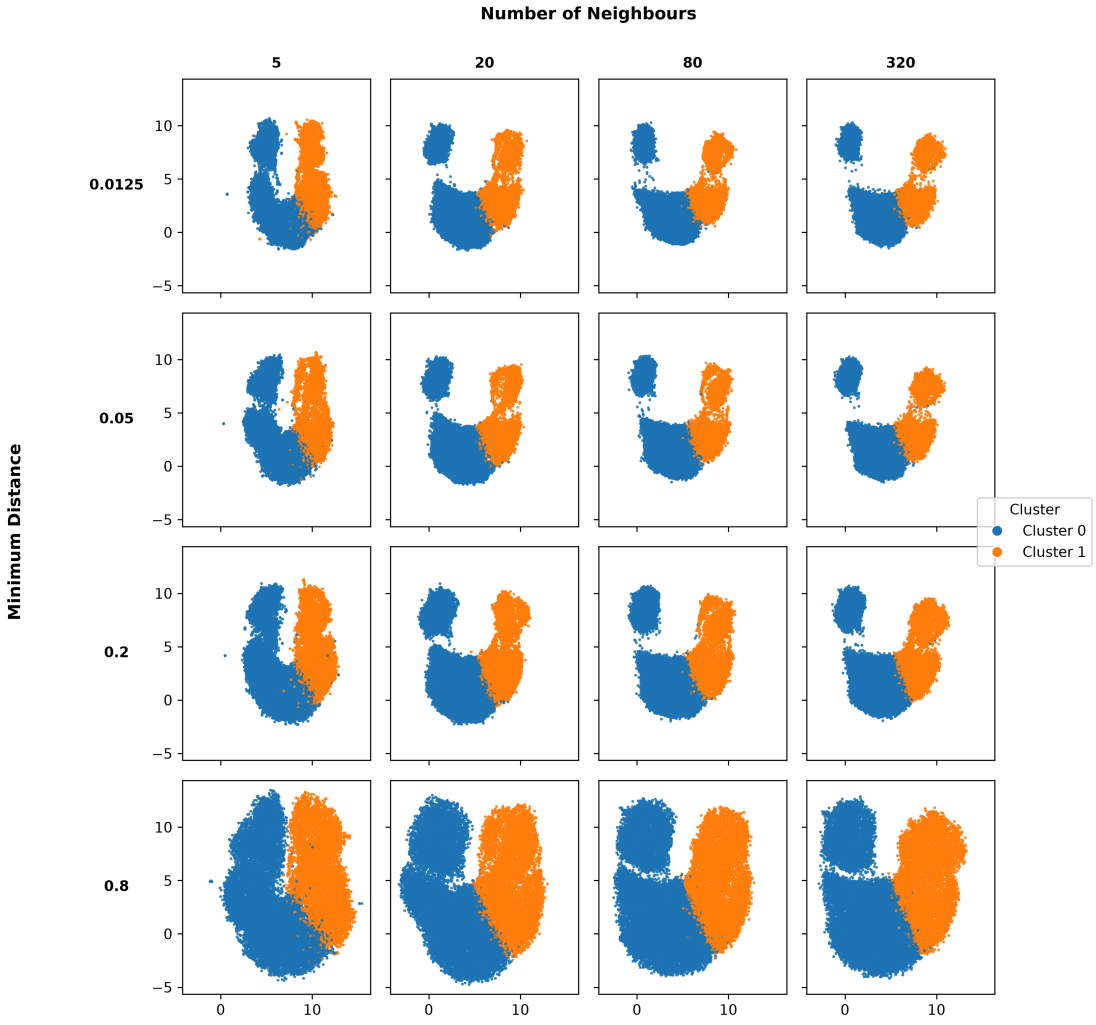


Figure 10: UMAP Coloured by K-means Cluster with Varying UMAP Hyperparameter Values

We can explore this potential within-cluster regionality via hierarchical clustering. Agglomerative clustering (Murtagh & Contreras, 2017) iteratively groups data instances together in pairs, whereby the initial data instance form pairs of instances, those pairs are then paired into groups, and so on until the dataset is unified under a single cluster. A number of measures can be used to determined which instances are paired together at each iteration. Ward linkage (Murtagh & Legendre, 2014) is one such measure, which pairs instances/groups in a manner which minimises the within-cluster sum of squared distances of the resultant merger. This has the effect of maximising compactness within each hierarchical cluster.

The derived pairings, which each minimise Ward linkage, from the agglomerative clustering algorithm can be plotted using a dendrogram, whereby each paring's associated Ward linkage value are plotted against the y-axis. This allows a visualisation of both the hierarchical cluster structure and a measure of by how much each clustering iteration increases the Ward linkage.

To aid computational efficiency, a subsample of $10,000$ normalised embeddings were used to produce a dendrogram, enabling inspection of the resulting hierarchical structure.

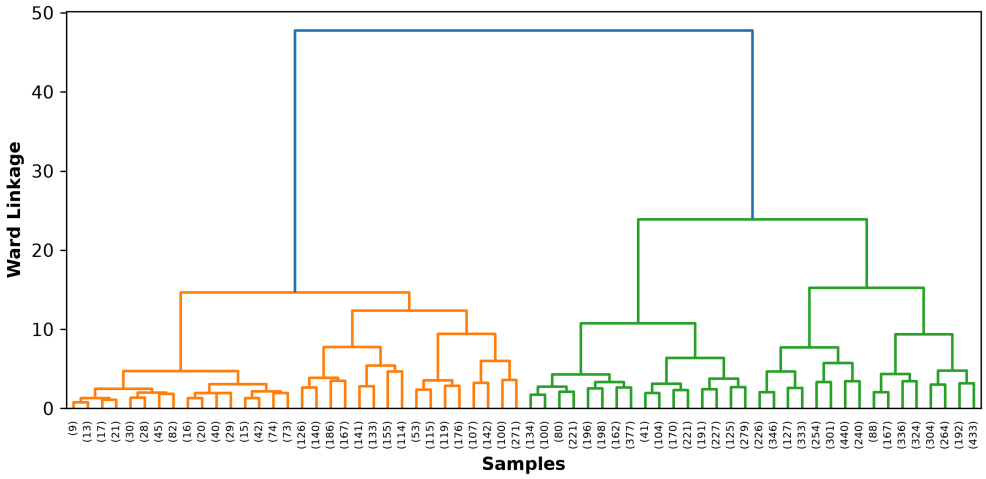


Figure 11: Agglomerative Clustering Dendrogram for Spatial Embeddings

Figure 11 appears to corroborate the interpretation of the UMAP and the results from k-means clustering, whereby we observe two broad clusters, but also evidence of each cluster containing 2 subclusters, as is suggested by the UMAP in Figure 10. These derived clusters from the agglomerative clustering algorithm are in moderately good agreement with those derived from the k-means algorithm, achieving an average adjusted rand index (ARI) of 0.564 across $1,000$ random samples each of size $10,000$. The sampling distribution can be seen below in Figure 12.

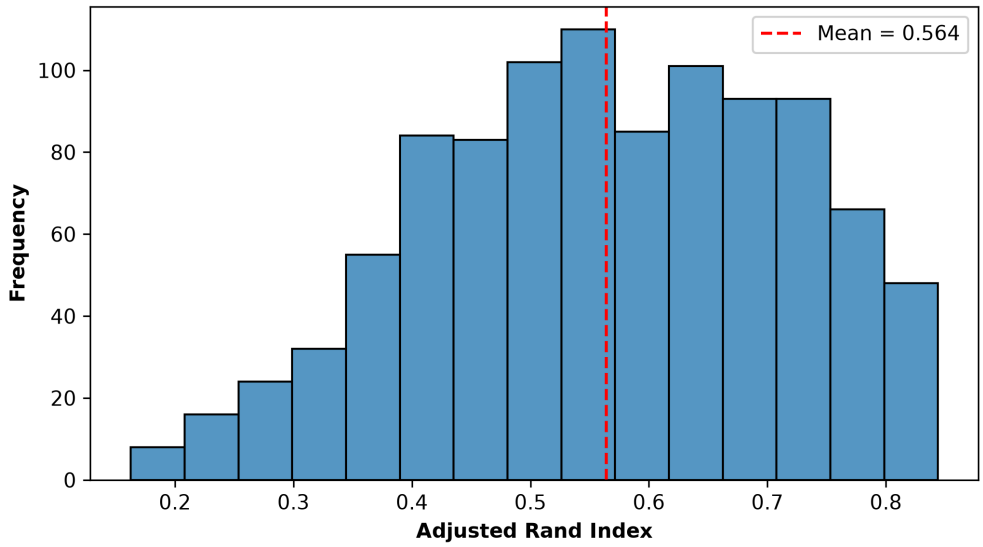


Figure 12: Adjusted Rand Index Sampling Distribution

5.1.3. *Spatial Embedding Dimension and Manually-Engineered Feature Correlations*

Embedding dimensions alone are not directly interpretable in terms of what constructs they represent, instead any present latent representations are typically spread out across multiple

embedding dimensions. However, we can gain some insight into what constructs might be represented within each embedding dimension by correlating them with more intepretable features.

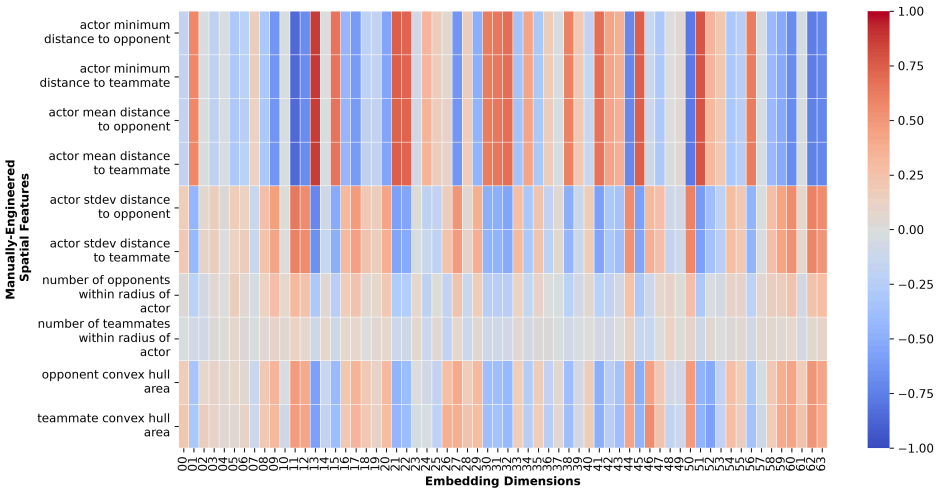


Figure 13: Manually-Engineered Spatial Features & Spatial Embeddings Correlation Heatmap

Figure 13 shows strong positive correlations between embedding dimensions 13, 45 and 51 and the 4 manually-engineered actor mean/minimum distances to teammates/opponents features, whereas embedding dimensions 11, 44 and 50 show strong negative correlations with these features. These embedding dimensions therefore seem to relate to spatial information regarding relative distances between the actor and other players.

Interesingly, it does not appear to be the case that any of the embeddings relate to the quantity of players within radius of the actor, with every dimension showing a negligible correlatons with these manually-engineered featrures.

Embedding dimensions 4, 10, 23, 37, 49 and 57 appear to have negligible correlations with all manually-engineered features, indicating that these dimensions potentially contain latent spatial information which is not captured by these manually-engineered features.

5.2. Classifier Training

Table 6 and Table 7 show the test results from training each of the 32 classifier models.

model	feature set	target	brier score	log loss	ece	auc
CatBoost	all	concedes	0.001421	0.009437	0.000924	0.871250
CatBoost	baseline	concedes	0.001455	0.009686	0.000885	0.873380
CatBoost	embeddings	concedes	0.001421	0.009498	0.000929	0.876378
CatBoost	manual	concedes	0.001466	0.009774	0.000926	0.868484
LR	all	concedes	0.001689	0.011276	0.000503	0.811814
LR	baseline	concedes	0.001713	0.011513	0.000488	0.799189
LR	embeddings	concedes	0.001707	0.011503	0.000511	0.797700
LR	manual	concedes	0.001690	0.011318	0.000526	0.811878
RF	all	concedes	0.001574	0.010644	0.000487	0.863532

model	feature set	target	brier score	log loss	ece	auc
RF	baseline	concedes	0.001558	0.010462	0.000498	0.858906
RF	embeddings	concedes	0.001572	0.010572	0.000498	0.869744
RF	manual	concedes	0.001566	0.010432	0.000492	0.866527
XGBoost	all	concedes	0.001407	0.008746	0.000545	0.897936
XGBoost	baseline	concedes	0.001427	0.008772	0.000538	0.902327
XGBoost	embeddings	concedes	0.001422	0.008728	0.000536	0.903058
XGBoost	manual	concedes	0.001405	0.008756	0.000540	0.895724

Table 6: Concedes Classifiers Test Results

For the concede classifiers, we can see that the XGBoost classifier with the baseline + embeddings feature set (i.e. *embeddings*) performed best in terms of AUC (0.903) and log loss (0.009). The random forest classifier which included all feature sets (i.e. *all*) showed superior calibration over other concede-based classifiers, with the baseline + manual feature set (i.e. *manual*) CatBoost classifier minimising Brier score.

Overall, the performance of concede classifiers showed a fairly wide range of variation in AUC (0.798–0.903) across the 4 different classifier techniques, however if the logistic regression classifiers are excluded, this range shrinks somewhat (0.859–0.903).

Logistic regression appeared to benefit most from the introduction of spatial context features, resulting in an increase in AUC from 0.799 to 0.812 with the baseline + manually-engineered spatial context feature set (i.e. *manual*). Despite this the *embeddings* feature set saw a very slight reduction in performance in terms of AUC, decreasing from 0.799 to 0.798.

By contrast, the 3 tree-based classifiers all saw a slight improvement in AUC due to the introduction of spatial embeddings and this improvment slightly exceeded that provided by the manual spatial feature set. Despite this, the additional computational overhead introduced by training spatial embeddings compared to creating engineered spatial features arguably does not justify the slight improvement in classifier performance.

model	feature set	target	brier score	log loss	ece	auc
CatBoost	all	scores	0.006927	0.035564	0.002102	0.877921
CatBoost	baseline	scores	0.006925	0.035232	0.002037	0.882362
CatBoost	embeddings	scores	0.006909	0.035437	0.002133	0.876865
CatBoost	manual	scores	0.006883	0.035251	0.001959	0.879218
LR	all	scores	0.007954	0.039307	0.000634	0.858193
LR	baseline	scores	0.007836	0.038097	0.001026	0.873419
LR	embeddings	scores	0.007787	0.037849	0.000823	0.875344
LR	manual	scores	0.007960	0.039565	0.000830	0.852066
RF	all	scores	0.007884	0.041154	0.001146	0.806775
RF	baseline	scores	0.007674	0.040200	0.000836	0.833511
RF	embeddings	scores	0.007824	0.040764	0.000969	0.811977
RF	manual	scores	0.007680	0.040376	0.000819	0.816687
XGBoost	all	scores	0.006818	0.033858	0.000640	0.891670
XGBoost	baseline	scores	0.006815	0.033761	0.000617	0.893760
XGBoost	embeddings	scores	0.006826	0.033874	0.000605	0.892156
XGBoost	manual	scores	0.006797	0.033632	0.000548	0.893949

Table 7: Scores Classifiers Test Results

For the score classifiers, the XGBoost classifiers dominate performance. The XGBoost classifier using *manual* features set achieved the best AUC (0.894), log loss (0.034) and ECE (0.001). Furthermore, the *baseline* feature set XGBoost classifier minimised Brier score most strongly (0.007).

Overall, the performance of score classifiers showed a slightly narrower range of variation in AUC (0.807–0.894) than with the concede classifiers. Interestingly, rather than the tree-based classifiers showing superior performance in terms of AUC, we can see that the random forest classifier performed worse across all feature sets than logistic regression.

We can see, for each tree-based classifier technique, the introduction of *embeddings* resulted in a reduction in classifier performance compared to the *baseline* feature set, whereas for logistic regression performance improved (0.873 to 0.875). Moreover, for CatBoost, logistic regression and random forest, the *manual* features resulted in reduction in performance, whilst for XGBoost performance appeared to slightly increase.

5.2.1. *Feature Set Significance Tests*

The results outlined in Table 6 and Table 7 only outline point estimates for classifier performance, and thus we lack sufficient evidence to determine whether the observed variation between classifiers may just be due to random variation. In line with the research objectives, we are particularly interested in whether the observed differences in AUC scores between classifiers using different feature sets *within the same classifier type (concede vs score) and technique* vary significantly. Given that each classifier technique within each type of classifier has 4 feature sets and we want to compare them pairwise, we have $\binom{4}{2}$ unique pairwise comparisons to make (i.e. 6).

5.2.1.1. *Concedes Classifier*

Table 8 to Table 11 shows no significant pairwise differences, indicating that the addition of spatial context yielded non-significant changes in classifier performance. We therefore lack sufficient evidence to suggest that spatial context improves any of the concede classifiers performance, yet we can also say there is insufficient evidence that spatial context yields a decrease in performance for each classifier technique.

Despite all pairwise differences being non-significant after Holm correction, the effect sizes, whilst small, do show increases in AUC between the *baseline* and *embeddings* feature sets for the tree-based classifiers, whereas logistic regression shows a slight decrease. Moreover, effect sizes suggest the *manual* feature set led to increases in AUC for the logistic regression and random forest concede classifiers, whilst decreases were observed for CatBoost and XGBoost.

Overall, it would therefore appear that the addition of spatial context features for the concede classifier would yield a very small improvment in classifier performance, but this improvment is mostly limited to the tree-based classifiers and fails to exceed the standard threshold of statistical significance.

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	0.24	1	2.13e-03	0.8712	0.8734
all vs embeddings	0.87	1	5.13e-03	0.8712	0.8764
all vs manual	−0.39	1	−2.77e-03	0.8712	0.8685
baseline vs embeddings	0.36	1	3.00e-03	0.8734	0.8764
baseline vs manual	−0.71	1	−4.90e-03	0.8734	0.8685

comparison	z	p holm	delta auc	auc 1	auc 2
embeddings vs manual	−1.03	1	−7.89e-03	0.8764	0.8685

Table 8: Concedes Model Feature Set Significance Test Comparison (CatBoost)

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	−1.31	0.8	−1.26e-02	0.8118	0.7992
all vs embeddings	−1.47	0.8	−1.41e-02	0.8118	0.7977
all vs manual	0.05	1	6.44e-05	0.8118	0.8119
baseline vs embeddings	−1.17	0.8	−1.49e-03	0.7992	0.7977
baseline vs manual	1.30	0.8	1.27e-02	0.7992	0.8119
embeddings vs manual	1.47	0.8	1.42e-02	0.7977	0.8119

Table 9: Concedes Model Feature Set Significance Test Comparison (Logistic Regression)

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	−0.82	1	−4.63e-03	0.8635	0.8589
all vs embeddings	1.49	0.5	6.21e-03	0.8635	0.8697
all vs manual	0.63	1	2.99e-03	0.8635	0.8665
baseline vs embeddings	2.22	0.2	1.08e-02	0.8589	0.8697
baseline vs manual	1.92	0.3	7.62e-03	0.8589	0.8665
embeddings vs manual	−0.90	1	−3.22e-03	0.8697	0.8665

Table 10: Concedes Model Feature Set Significance Test Comparison (Random Forest)

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	1.36	0.5	4.39e-03	0.8979	0.9023
all vs embeddings	2.14	0.2	5.12e-03	0.8979	0.9031
all vs manual	−0.84	0.8	−2.21e-03	0.8979	0.8957
baseline vs embeddings	0.20	0.8	7.32e-04	0.9023	0.9031
baseline vs manual	−1.97	0.2	−6.60e-03	0.9023	0.8957
embeddings vs manual	−2.19	0.2	−7.33e-03	0.9031	0.8957

Table 11: Concedes Model Feature Set Significance Test Comparison (XGBoost)

5.2.1.2. *Scores Classifier*

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	1.53	0.6	4.44e-03	0.8779	0.8824
all vs embeddings	−0.52	1	−1.06e-03	0.8779	0.8769
all vs manual	0.47	1	1.30e-03	0.8779	0.8792
baseline vs embeddings	−1.91	0.3	−5.50e-03	0.8824	0.8769
baseline vs manual	−1.26	0.8	−3.14e-03	0.8824	0.8792

comparison	z	p holm	delta auc	auc 1	auc 2
embeddings vs manual	0.79	1	2.35e-03	0.8769	0.8792

Table 12: Scores Model Feature Set Significance Test Comparison (CatBoost)

Table 12 shows none of the observed pairwise differences to be statistically significant for the CatBoost score classifier model. Effect sizes show the introduction of the *manual* ($z = -1.26$, Holm-adjusted $p = .800$) and *embeddings* ($z = -1.91$, Holm-adjusted $p = .300$) spatial context features result in a slight reduction in classifier performance compared to the *baseline* feature set. This reduction is minutely more pronounced for the *embeddings* feature set, suggesting the *manual* feature set provided slightly more useful predictive information than the *embeddings*, despite both hurting classifier performance.

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	6.59	1e-10	1.52e-02	0.8582	0.8734
all vs embeddings	7.06	7e-12	1.72e-02	0.8582	0.8753
all vs manual	−6.14	2e-09	−6.13e-03	0.8582	0.8521
baseline vs embeddings	3.03	0.002	1.92e-03	0.8734	0.8753
baseline vs manual	−7.95	9e-15	−2.14e-02	0.8734	0.8521
embeddings vs manual	−8.35	4e-16	−2.33e-02	0.8753	0.8521

Table 13: Scores Model Feature Set Significance Test Comparison (Logistic Regression)

Table 13 outlines DeLong test results for the logistic regression score classifier. We can observe that all pairwise differences are statistically significant following the Holm correction. Whilst the *manual* feature set showed a reduction in performance compared to the *baseline*, the *baseline vs embeddings* comparison showed a significant improvment in classifier performance for the *embeddings* feature set over the *baseline* ($z = 3.03$, Holm-adjusted $p = .002$). Whilst the effect size is quite small (Δ AUC $= 0.00192$), this result does provide evidence that the *embeddings* added useful information for scoring classifications for the logistic regression model, beyond what was represented by the manually-engineered features which hurt classifier performance.

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	8.19	2e-15	2.67e-02	0.8068	0.8335
all vs embeddings	2.22	0.05	5.20e-03	0.8068	0.8120
all vs manual	3.96	0.0002	9.91e-03	0.8068	0.8167
baseline vs embeddings	−7.34	1e-12	−2.15e-02	0.8335	0.8120
baseline vs manual	−5.48	2e-07	−1.68e-02	0.8335	0.8167
embeddings vs manual	1.51	0.1	4.71e-03	0.8120	0.8167

Table 14: Scores Model Feature Set Significance Test Comparison (Random Forest)

Regarding the random forest score classifier, Table 14 again shows 5 of the 6 pairwise differences to be statistically significant. Both the *manual* ($z = -5.48$, Holm-adjusted $p < .001$) and *embeddings* ($z = -7.34$, Holm-adjusted $p < .001$) feature sets resulted in a significant reduction

in classifier performance compared to the *baseline*. In light of the *embeddings vs manual* comparison yielding a non-significant difference ($z = 1.51$, Holm-adjusted $p = .100$), we lack sufficient evidence to determine which spatial context feature set hurts the performance of this classifier more.

comparison	z	p holm	delta auc	auc 1	auc 2
all vs baseline	2.70	0.03	2.09e-03	0.8917	0.8938
all vs embeddings	0.78	0.9	4.86e-04	0.8917	0.8922
all vs manual	3.55	0.002	2.28e-03	0.8917	0.8939
baseline vs embeddings	−2.30	0.09	−1.60e-03	0.8938	0.8922
baseline vs manual	0.27	0.9	1.89e-04	0.8938	0.8939
embeddings vs manual	2.14	0.1	1.79e-03	0.8922	0.8939

Table 15: Scores Model Feature Set Significance Test Comparison (XGBoost)

Lastly, Table 15 outlines DeLong test results for the XGBoost score classifier. 2 of the 6 pairwise comparisons made were statistically significant, whereby the *baseline* feature set significantly exceeds performance of the XGBoost classifier using *all* feature sets ($z = 2.70$, Holm-adjusted $p = .030$). The significant *all vs manual* comparison ($z = 3.55$, Holm-adjusted $p = .002$) and the non-significant *baseline vs manual* comparison ($z = 0.27$, Holm-adjusted $p = .900$) suggest the *embeddings* feature set hurts classifier performance, whilst the *manual* feature set does not yield a signficant change in performance over the baseline.

5.2.2. *Classifiers Used for VAEP in this Project*

Given the results outlined in Section 5.2, Section 5.2.1.1 and Section 5.2.1.2, the overall best performing classifier techniques was XGBoost, for both scoring and conceding predictions. Specifically, the *embeddings* XGBoost concede classifier and the *manual* features XGBoost score classifier achieved the best AUC scores, despite the performance of these feature sets not being significantly different from their respective *baseline* counterparts. However, on this basis, the classifiers used to produce VAEP scores are these two classifiers.

5.2.3. *Valuing Actions by Estimating Probabilities (VAEP) EDA*

As seen in Decroos, Bransen, Van Haaren, & Davis (2019), it may also be useful to assess how derived VAEP values behave and align with domain-specific intuition. It is desireable for derived metrics in any domain to display strong face validity, that is, the metric should provide a well-calibrated and "plausible" (Jacobs & Wallach, 2021, pp. 5) measure of the phenomenon it is intended to capture. In measuring action value, one inherent limitation is that action-valuation metrics like VAEP lack a "ground truth" upon which to draw comparisons (Decroos et al., 2020). There is no objective measure of action value that we can observe in the world beyond what is provided by the very action-valuation metrics researchers have developed. Given that such metrics are intended to provide a quantitative measure of action value, assessing face validity is fraught with subjectivity and it is thus somewhat trivially obvious that this issue arises.

5.2.3.1. *"The Messi Test"*

One heuristic used in sports analytics is to compare derived action or player valuations with the intuitions of both technical and non-technical domain experts. For instance, Pappalardo et al.

(2019) calibrate their PlayeRank metric by assessing its agreement with a panel of experienced football scouts. Regarding the period of time which publicly available events data, such as those provided by StatsBomb, covers, it is widely regarded that players such as *Lionel Messi* and *Atiana Bonmati* constitute one of the most, if not the most, valuable male and female players in world football, respectively (For example, they both lead the all-time most number of Ballon d'Or wins by a single player for their respective gender [8 and 3, respectively], Wikipedia (2026a); Wikipedia (2026b)). Dubbed "the Messi test", the heuristic suggests that an action value metric displays strong face validity if it correctly rates players who are widely regarded as being exceptionally talented as highly performing.

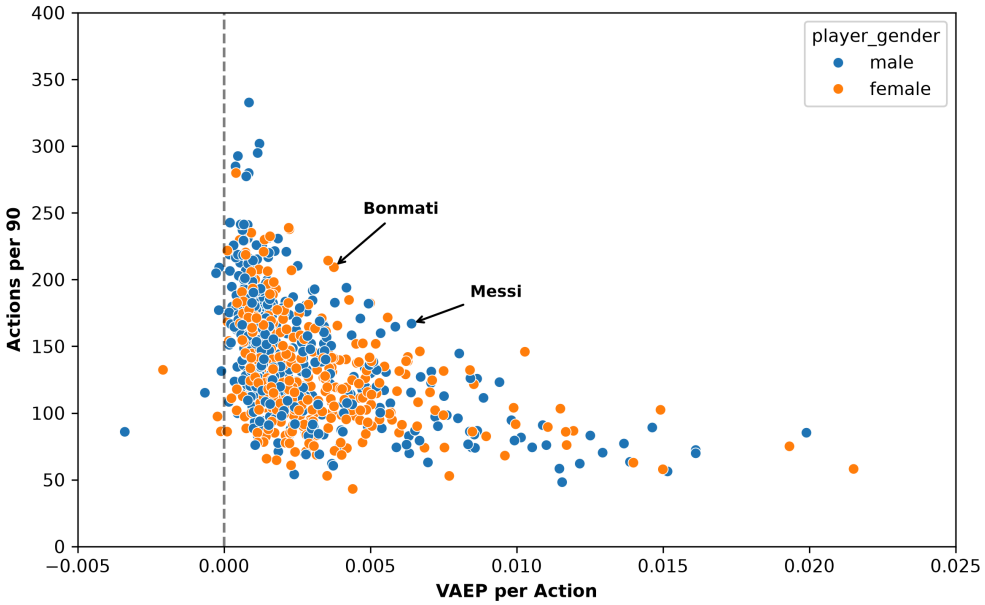


Figure 14: VAEP per Action vs Actions per 90 Minutes Played

We can see from Figure 14 that our "Messi/Bonmati test" appears to have been passed successfully. The plot includes players who have player more than 270 minutes across the competitions included in this project's dataset and compares the number of actions they perform per 90 minutes of playtime with the average VAEP value of the set of actions they took. We would expect more valuable players to exhibit both high volume (actions per 90) and high quality (VAEP per action) across their playtime and action sets. We can see that both *Messi* and *Bonmati* are positioned on the upper frontier of the scatter plot, exactly where valuable players would be expected to appear. Furthermore, the derived VAEP per action values fall within the same magnitude as that observed in Decroos, Bransen, Van Haaren, & Davis (2019), indicating some degree of agreement in valuation with previous VAEP models. The number of actions is likely generally higher than seen in Decroos, Bransen, Van Haaren, & Davis (2019) as this project uses atomic-SPADL actions, rather than standard SPADL actions.

5.2.3.2. *VAEP Player Ratings & Player Market Value*

Using the VAEP player rating fomrula provided in Equation 3, we can assign ratings to each player in our dataset. Table 16 and Table 17 outline the top-10 VAEP player ratings for male and female footballers, respectively. Whilst *Bonmati* does not appear in the women's top-10, presumably due to their lower VAEP per action score, *Messi* does appear and with considerably

higher minutes played than the rest. Each table is dominated by players who play in attacking positions, with *Hinata Miyazawa* being the only player to not do so.

player	minutes_played	market_value_in_eur	rating
Niclas Füllkrug	276.6	€3,500,000.00	1.696428
Patrik Schick	533.0	€18,000,000.00	1.304686
Wout Weghorst	448.3	€2,000,000.00	1.164884
Mikel Oyarzabal Ugarte	363.5	€25,000,000.00	1.162477
Karim Benzema	351.6	€5,000,000.00	1.158929
Richarlison de Andrade	327.3	€25,000,000.00	1.125808
Emil Peter Forsberg	376.6	€1,800,000.00	1.087393
Lionel Andrés Messi Cuccittini	732.0	€15,000,000.00	1.070157
Salem Mohammed Al Dawsari	297.0	€1,200,000.00	1.061486
Kasper Dolberg	401.4	€7,000,000.00	1.053966

Table 16: Top-10 VAEP Player Ratings (Male)

player	minutes_played	rating
Alexandra Popp	664.4	1.527324
Hinata Miyazawa	349.7	1.500218
Sophie Roman Haug	281.7	1.450418
Marie-Antoinette Katoto	398.0	1.250226
Alba María Redondo Ferrer	414.8	1.186992
Mina Tanaka	273.0	1.111318
Bethany Mead	708.3	1.038069
Esther Gonzalez Rodríguez	829.8	1.034848
Cristiana Girelli	610.4	1.028786
Thembi Kgatlana	377.1	1.003194

Table 17: Top-10 VAEP Player Ratings (Female)

Decroos, Bransen, Van Haaren, & Davis (2019) also explore how their derived VAEP player ratings relate to the market value of players. The authors report the top-10 rated Premier League players from the 2017/18 season along with their VAEP player ratings for that season (see Table 1d in their article). For our present data, only male football have sufficient market value data available to use, and this was collected from Transfermarkt (Transfermarkt, n.d.), using the Reep register (Reep, n.d.) to ensure player valuations were assigned to the correct StatsBomb player IDs. Player valuations represent their most recent recorded market valuation since 01/08/2025.

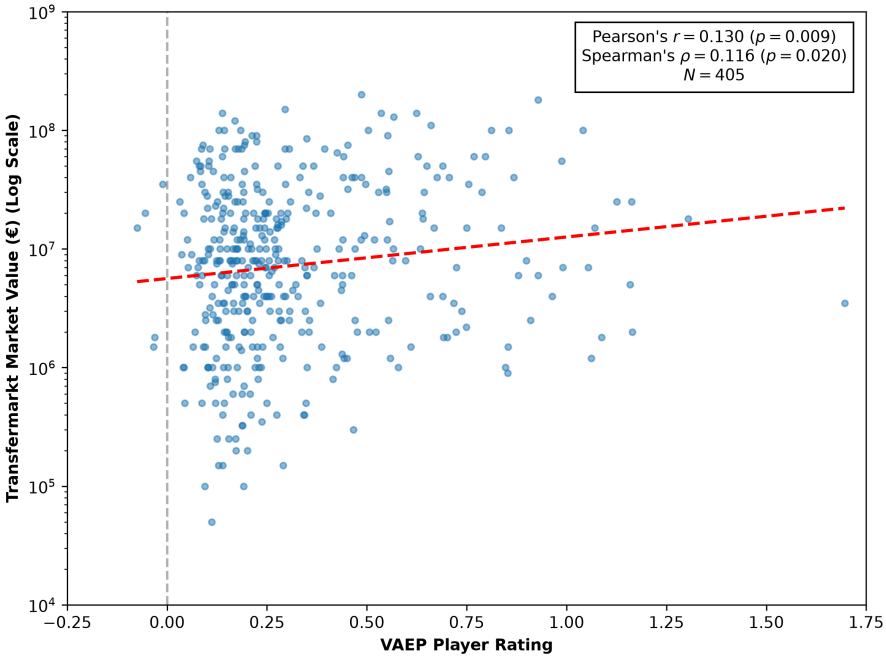


Figure 15: VAEP Player Ratings vs Transfermarkt Market Value

Whilst market valuation is not entirely determined by the average value of the players set of actions of some time interval, Figure 15 does affirm our intuition that players who provide more valuable actions are generally valued more highly in the market, whereby a small, statistically significant, positive Spearman's rank correlation was observed between VAEP player rating and logarithmic Transfermarkt market valuation, $\rho = .116, p = .02, N = 405$.

5.3. Model Interpretation

5.3.1. *Permutation Importance*

Beyond assessing the performance and face validity of the VAEP classifier models, we may also attempt to interpret which features are most influential in the decisions made by these classifiers. By exploring permutation importance, we can accumulate evidence to indicate which features are most impactful in enabling strong classification performance.

5.3.1.1. *Grouped Permutation Importance*

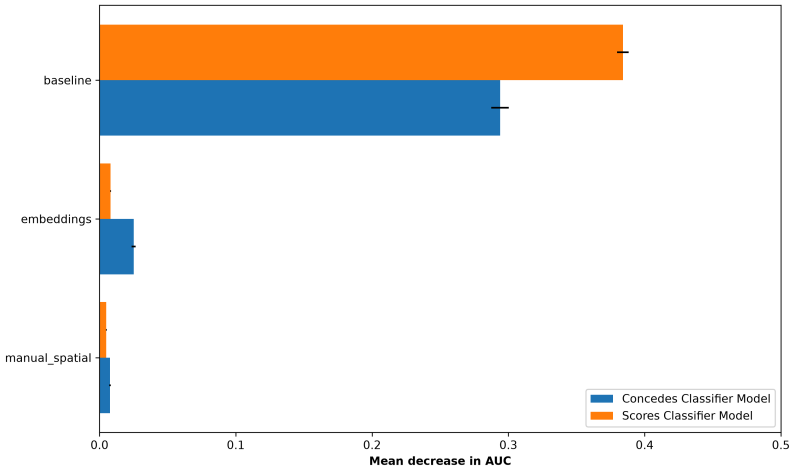


Figure 16: Grouped Permutation Importance

Figure 16 shows the average decrease in AUC (across 10 iterations) due to the random permutation of *baseline* features, spatial embeddings (*embeddings*) and manually-engineered spatial features (*manual*).

For the selected concede and score classifiers (see Section 5.2.2), the *baseline* feature set dominates as most important for determining the classifier's predictions. Due to random permutations of these *baseline* features, AUC, on average, reduces by around 0.3 for the concede classifier and 0.38 for the score classifier.

By contrast, and matching the pattern observed in Section 5.2.1, the random permutation of the *embeddings* and *manual* feature sets do not yield a particularly large average reduction in AUC. However, we can observe that the permutation importance of the *embeddings* feature set is larger for the concede classifier than for the score classifier, which is in line with the pattern of observations made in Section 5.2.1.1, where we observed a non-significant increase in the concede XGBoost classifier performance upon the addition of the *embeddings* feature set to the *baseline*, whilst this addition instead hurt model performance for the score XGBoost classifier, albeit non-significantly.

5.3.1.2. *Baseline Feature Set Permutation Importance*

We can zoom in on each individual feature which constitutes the *baseline* feature set to examine which carries importance for each prediction task (score or concede). Given that each feature in the model has an additional 9 lagged variants (in order to reflect the $k = 10$ previous actions), each feature will be very highly correlated with its own lagged variants. As such, it would be more appropriate to group together each feature with its lagged variants before evaluating the permutation importance of each feature. This evaluation uses the XGBoost concede and score models trained on the *baseline* feature set only.

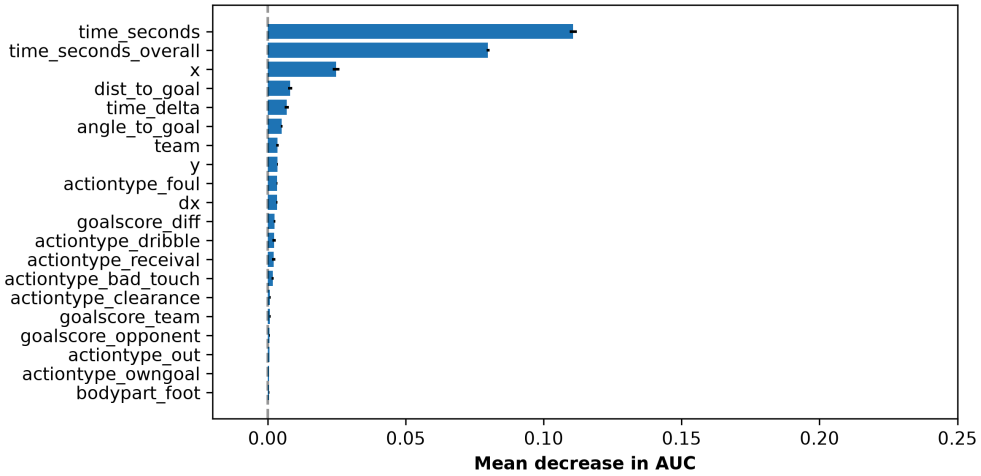


Figure 17: Concede Classifier Permutation Importance: Baseline Features

Figure 17 reveals the current time (in seconds) of the set of actions (i.e. the gamestate) within the $k = 10$ window of the current action to be the most important features. We can also see that the `x` co-ordinate of the gamestate and its distance to the opponent's goal (`dist_to_goal`) also appear to be important features. It would appear then that much of the concede XGBoost *baseline* classifier performance can be attributed to knowning the time at which an action occurred, along with its x co-ordinate or Euclidean distance to the opponent's goal. This may be somewhat unsuprising, as we might intuitively expect the probability of an action resulting in conceding a goal to depend on where on the pitch and when during the match the action occurred.

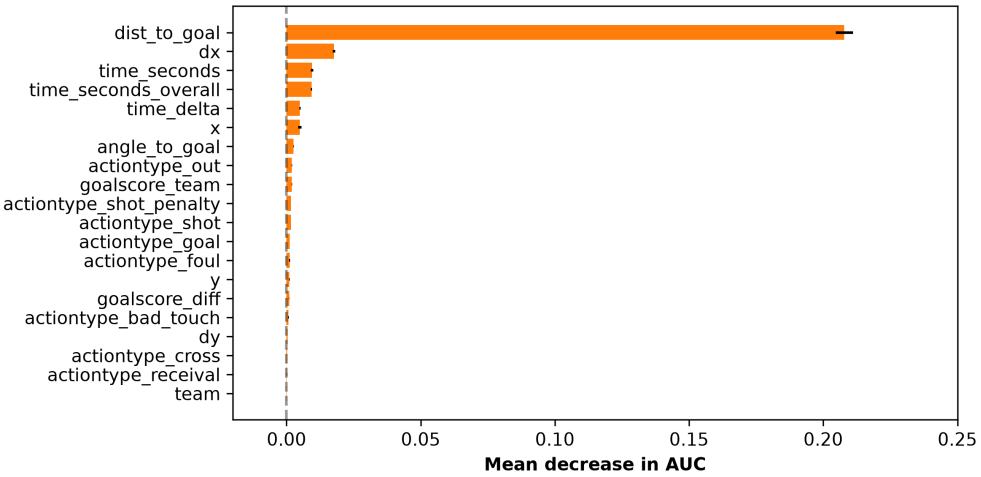


Figure 18: Score Classifier Permutation Importance: Baseline Features

Figure 18 shows some similarities in *baseline* feature importance for the score classifier as the concede classifier. Again, we observe timestamp and distance to goal features to be amongst the most important. However, we can see that the `dist_to_goal` of the gamestate appears far more important than any other *baseline* feature for predicting a goal being scored within the next k actions. Again, this both matches intuition, as we might expect goal scoring to be easier the closer a player is to the opponent's goal, and empirical probablities estimated by alternative action-valuation models like expected goals (xG) or expected threat (xT).

5.3.1.3. *Manually-Engineered Feature Set Permutation Importance*

We can additionally examine the importance of the manually-engineered features by exploring their permutation importance as part of the concede and score XGBoost *manual* feature set classifiers. We should note that, for each of these plots, the scale of the x-axis is around $10\times$ smaller than that of the *baseline* feature performance plots and so these observed importances are still relatively minuscule.

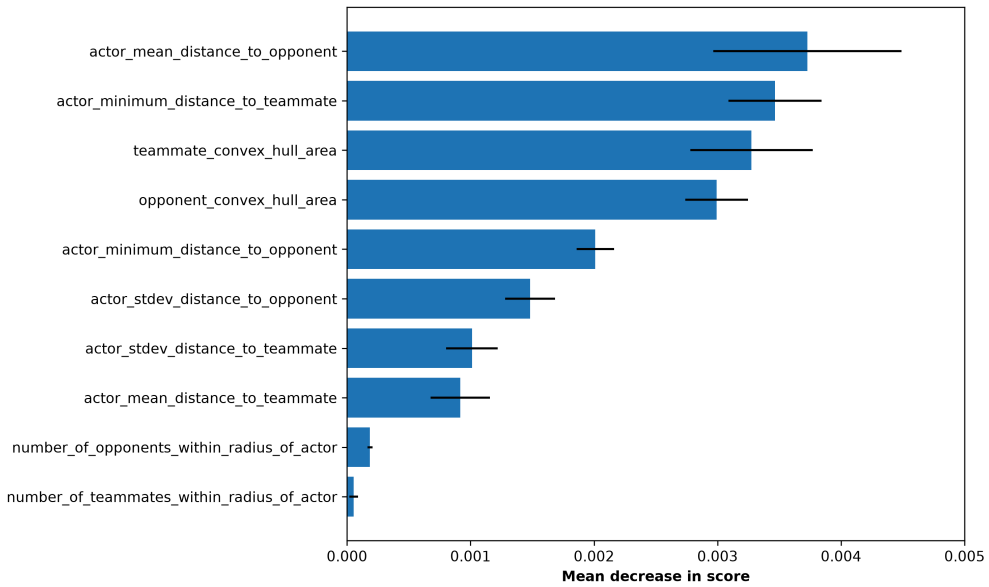


Figure 19: Concede Classifier Permutation Importance: Manually Engineered Spatial Features

Figure 19 shows that the average distance of the actor to the opponents, their minimum distance to a teammate and the convex hull areas of both teammates and opponents to be the most important manually engineered features. Spatial context of how close the actor is to opponents, if they are close to a teammate and the how spread out teammate and opponent formations are each contribute most to estimating concede probabilities.

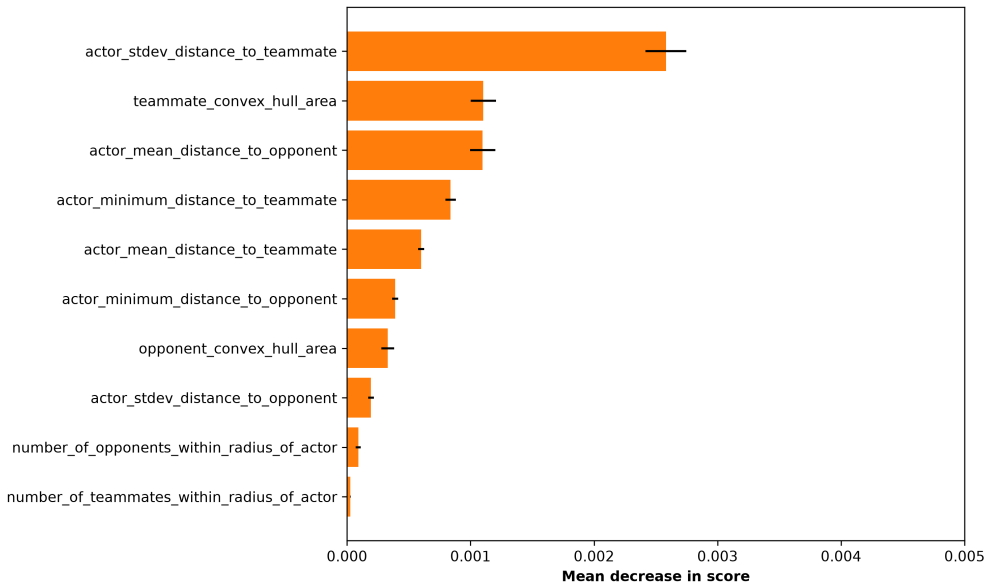


Figure 20: Score Classifier Permutation Importance: Manually Engineered Spatial Features

By contrast, Figure 20 shows the standard deviation of the distances between the actor and their teammates to be the most important manually-engineered spatial feature for the score classifier. This would suggest the most useful spatial context for estimating scoring probabilities revolves around how much variation there is in distances between the actor and their teammates. Such a finding might indicate that a more or less structured attacking shape changes the probability of scoring within k events, where less variation in teammate distances would indicate a more organised team structure than might be observed if there is large variation in the distances of teammates to the actor.

5.3.1.4. *Spatial Embedding Dimensions Permutation Importance*

Figure 21 and Figure 22 outlines which embedding dimensions were most important for each type of classifier. We can see that for the concede classifier, embedding dimensions 26, 48 and 24 constitute the top 3 most important, with 46, 52 and 18 being most important for the score classifier.

With reference to Figure 13, the top 3 concede classifier embedding dimensions show slight positive correlations with relative actor to teammate/opponent distance metrics (dimension 24), slight negative correlations with teammate/opponent convex hull area (dimensions 26 & 48) and slight positive correlations with the number of teammate/opponents within 10m of the actor (dimension 48). The top 3 score embedding dimensions relate somewhat to relative actor to teammate/opponent distance metrics (negative for dimension 18, positive for dimension 52), moderate relationships with teammate/opponent convex hull area (positive for dimension 46, negative for dimension 52). The embedding dimensions with the strongest correlations to manually-engineered spatial features do not appear to be particularly important for the concede classifier, but are moreso for the score classifier, whereby dimensions 11, 13, 45 and 50 rank amongst the top-20 most important embedding dimensions.

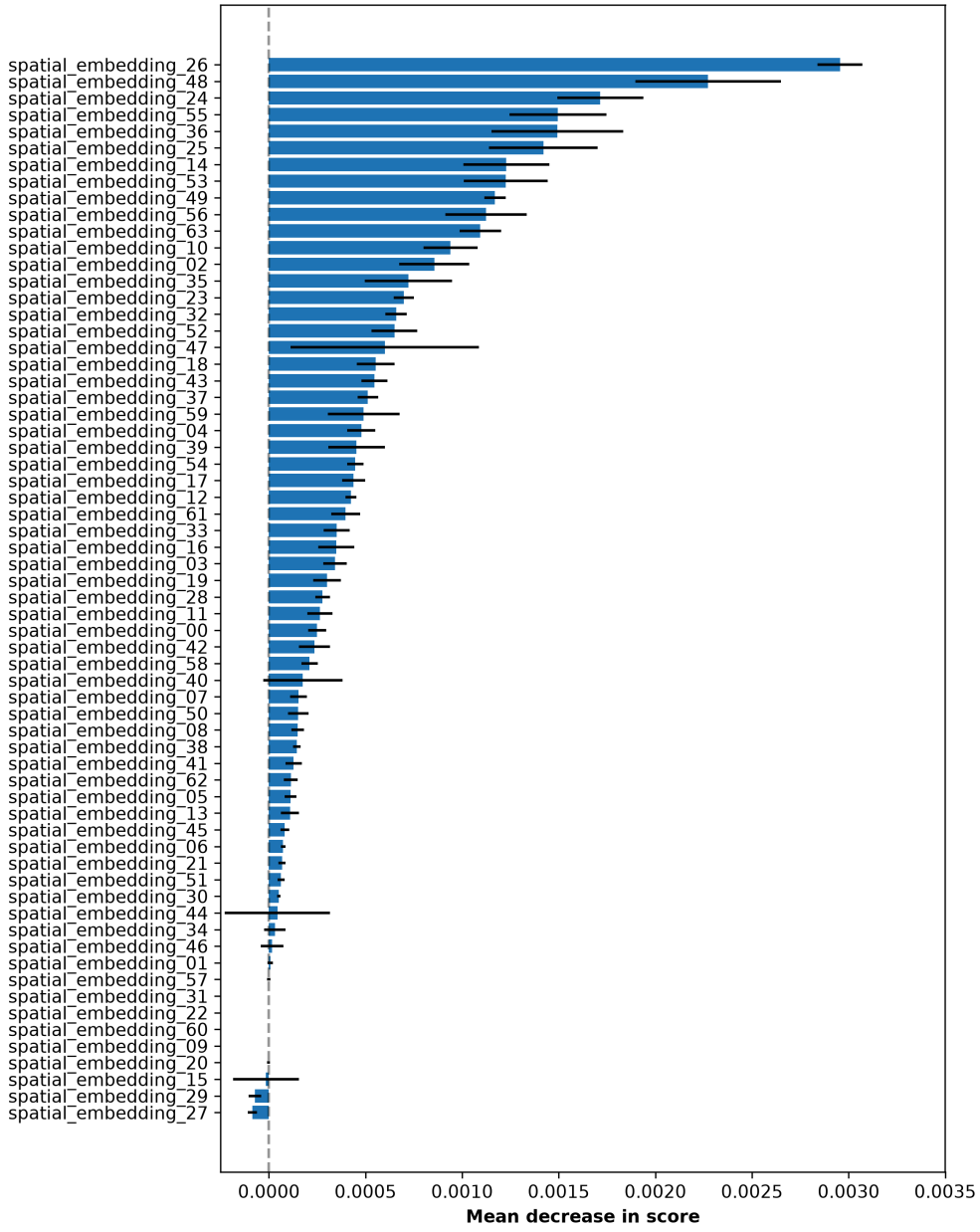


Figure 21: Concede Classifier Permutation Importance: Spatial Embedding Dimensions

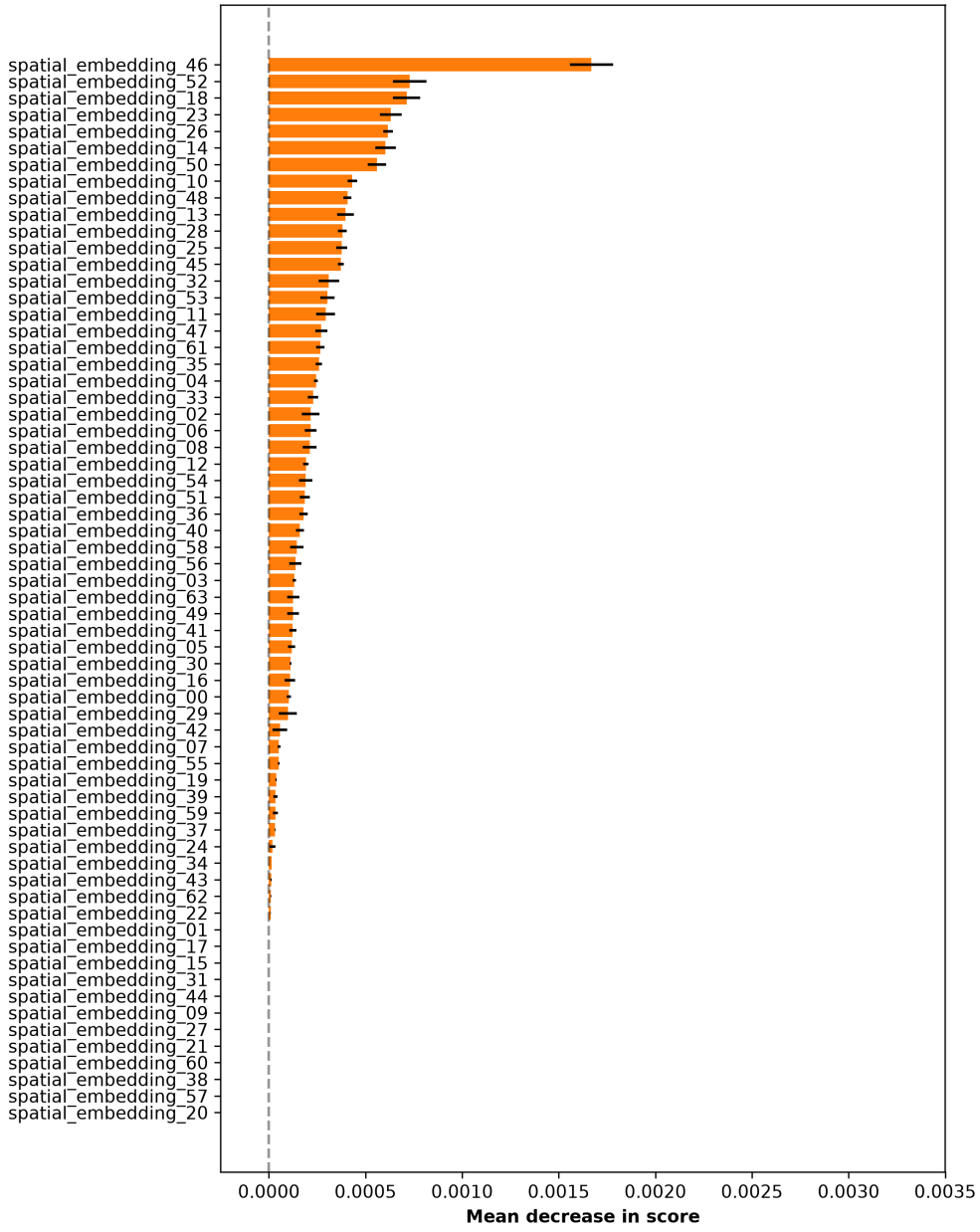


Figure 22: Score Classifier Permutation Importance: Spatial Embedding Dimensions

5.3.2. *Partial Dependence Plots*

Through partial dependence plots, we can explore how the predicted probability of scoring or conceding within k actions changes as we vary single feature.

As identified earlier, the most important baseline features for the concede and score classifiers were the `time_seconds` and `dist_to_goal` features, respectively. Figure 23 shows their respective partial dependencies.

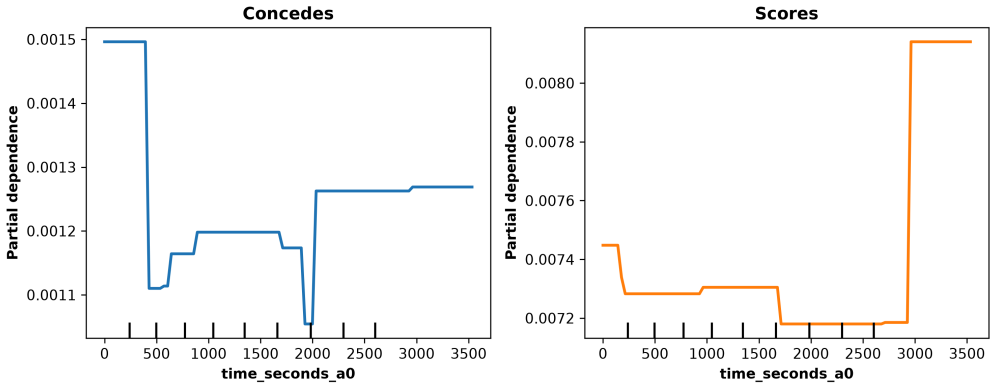


Figure 23: PDP Comparison of Most Important Concede Classifier Baseline Feature

The concede classifier plot shows the probability of conceding is higher during the first 450 seconds (5 minutes) of a half (`time_seconds` is the time elapsed in the current half, not the absolute time elapsed) compared to the rest of the half, where after $2,000$ (around the 33rd minute of the half) the probability increases again.

The score classifier plot shows an alternative pattern. Whilst the overall probability estimates are much higher than with the concede classifier, it appears that the predicted probability of scoring only slightly decreases over the course of each half, with the jump in probability occurring during first and second half stoppage time periods (i.e. $> 3,000$ seconds).

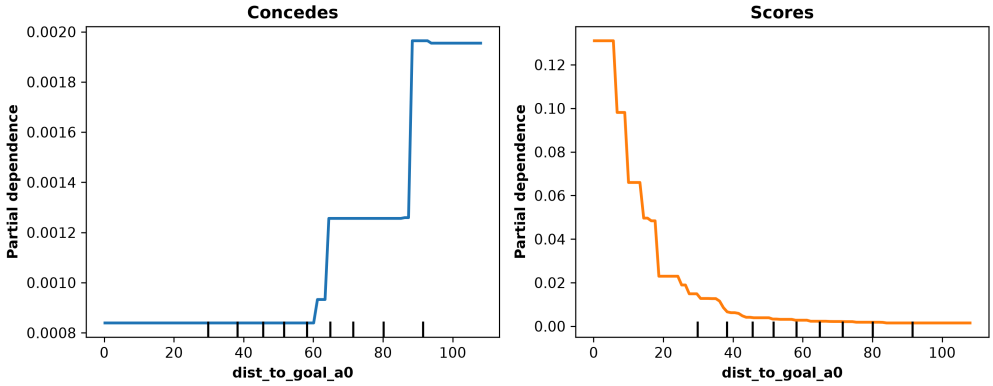


Figure 24: PDP Comparison of Most Important Score Classifier Baseline Feature

Regarding `dist_to_goal`, we can see patterns in Figure 24 which largely match our intuitions. For the concede classifier, the estimated probability of conceding increases exponentially as the distance of the action from the opponent's goal increases (i.e. as the action gets closer to the actor's own goal). Moreover, the score classifier shows the opposite pattern, where by as the distance from goal increases, the estimated probability of scoring within k events decreases exponentially. This again generally aligns with the pattern of action valuation provided by alternative models like xG and xT, which both assign higher action value to actions closer to the opponent's goal.

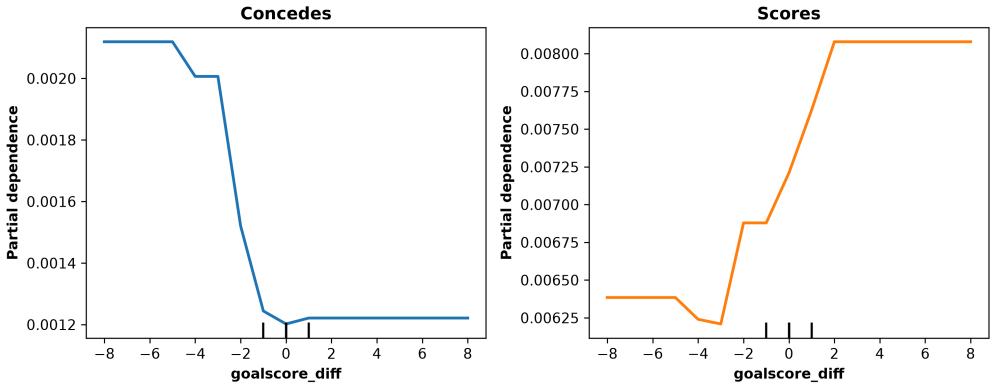


Figure 25: PDP of Goalscore Difference

A further sense-check can be observed in Figure 25, whereby the estimated probability of conceding is higher when the actor's team is losing, with that probability increasing exponentially, whilst the estimate probability remains constant as the goalscore difference increases above 0. The score classifier again shows a mostly opposite pattern, whereby the estimated probability of scoring remains low until the goalscore difference reaches −3, upon which we see an exponential increase until the goalscore difference reaches 2 and the estimated probability plateaus.

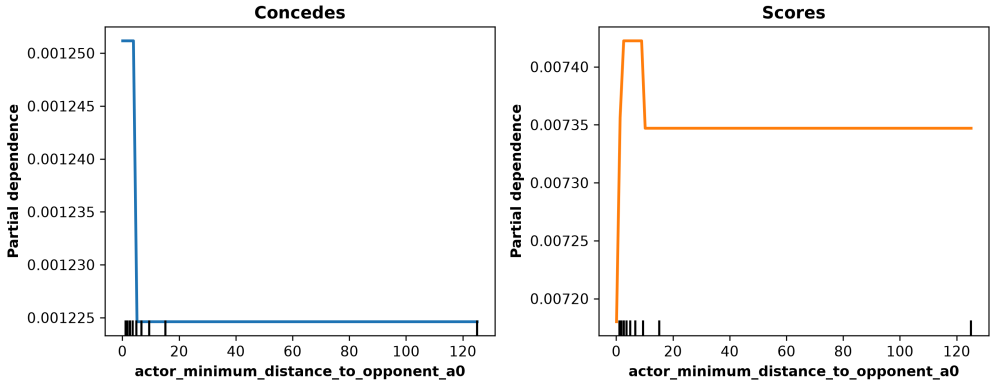


Figure 26: PDP of Actor's Distance to Nearest Opponent

Inspecting Figure 26 showing the PDP of the actor's minimum distance to opponents, using the *manual* feature set classifiers, we can see again a mostly inverted relationship across the concede and score classifiers. The concede classifier suggests the predicted probability of conceding exponentially decreases once there are no opponents within around 10 meters of the actor, which is very intuitive. Likewise, the score classifier shows a sharp increase in the probability of scoring as the minimum opponent distance increases, plateauing at around 15 meters. It should be noted that the scale of the y-axis is quite small, so these observed changes, whilst large in a relative sense, do not correspond to large absolute probabilities. Furthermore, the black ticks on the x-axis indicate the distribution of quantiles of the dataset, where we can see most of the data is congregated around the sharp changes in probability. The remaining plateaus are thus likely representative of a very small sample of actions which the classifiers assign roughly equal probabilities to.

6. Discussion

This project aimed to explore how the addition of spatial context features altered, if at all, the performance and behaviour of the VAEP Model using the full sample of events from 6 UEFA and FIFA tournaments across Men's (3) and Women's (3) football. By engineering feature representations of the spatial context surrounding the actor during a given action, using StatsBomb's 360 freeze-frame data, the standard SPADL feature set typically used in the construction of VAEP models was accompanied and the change in performance and behaviour assessed.

Regarding classifier performance, we observed in Section 5.2 an ablation design which compared the performance each of the 4 feature sets across each classifier technique for both the concede classifier and score classifier, respectively: the traditional SPADL feature set (*baseline*); baseline + manually-engineered spatial features (*manual*); baseline + spatial embeddings (*embeddings*); and baseline + manually-engineered spatial features + spatial embeddings (*all*).

7. Conclusion

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019, July 25). *Optuna: A Next-generation Hyperparameter Optimization Framework*. https://doi.org/10.48550/arXiv.1907.10902

Amichay, G., Silva, H., Brito, J., & Marcelino, R. (2025). Characterizing the Spatial Structures of Competing Football Teams. *Scientific Reports*, *15*(1), 35217–35218. https://doi.org/10.1038/s41598-025-97765-y

Anzer, G., & Bauer, P. (2021). A Goal Scoring Probability Model for Shots Based on Synchronized Positional and Event Data in Football (Soccer). *Frontiers in Sports and Active Living*, *3*, 624475–624476. https://doi.org/10.3389/fspor.2021.624475

Ashcroft, A., Das, A., Gryaditskaya, Y., Qu, Z., & Song, Y.-Z. (2023, October 13). *Modelling Complex Vector Drawings with Stroke-Clouds*. The Twelfth International Conference on Learning Representations. https://openreview.net/forum?id=O2jyuo89CK

Bassek, M., Rein, R., Weber, H., & Memmert, D. (2025). An Integrated Dataset of Spatiotemporal and Event Data in Elite Soccer. *Scientific Data*, *12*(1), 195–196. https://doi.org/10.1038/s41597-025-04505-y

Bergstra, J., & Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. *The Journal of Machine Learning Research*, *13*(null), 281–305. https://dl.acm.org/doi/10.5555/2188385.2188395

Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for Hyper-Parameter Optimization. *Advances in Neural Information Processing Systems*, *24*. https://papers.nips.cc/paper_files/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html

Bischofberger, J., Ma, R., Bauer, P., Arnsmeyer, K., & Baca, A. (2026, June 18). *Blame Is Easier than Praise: Measuring off-Ball Defensive Performance in Football*. https://doi.org/10.48550/arXiv.2606.19931

Breiman, L. (2001). Random Forests. *Machine Learning*, *45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Brier, G. W. (1950). Verification of Forecasts Expressed in Terms of Probability. *Monthly Weather Review*, *78*(1), 1–3. https://doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2

Burriel, B., & Buldú, J. M. (2021). The Quest for the Right Pass: Quantifying Player's Decision Making. *Proceedings of the StatsBomb Innovation in Football Conference*, 1–22. https://statsbomb.wpengine.com/wp-content/uploads/2021/11/Javier-M-Buldu.pdf

Calabuig, J. M., Catalán, C., García-Raffi, L. M., & Sánchez-Pérez, E. A. (2024). A Mathematical Model to Study Defensive Metrics in Football: Individual, Collective and Game Pressures. *Mathematics*, *12*(23), 3854–3855. https://doi.org/10.3390/math12233854

Caliński, T., & Harabasz, J. (1974). A Dendrite Method for Cluster Analysis. *Communications in Statistics*, *3*(1), 1–27. https://doi.org/10.1080/03610927408827101

Capellera, G., Ferraz, L., Rubio, A., Agudo, A., & Moreno-Noguer, F. (2024). Footbots: A Transformer-Based Architecture for Motion Prediction in Soccer. *2024 IEEE International Conference on Image Processing (ICIP)*, 2313–2319. https://doi.org/10.1109/ICIP51287.2024.10647396

Caron, M., Bojanowski, P., Joulin, A., & Douze, M. (2019, March 18). *Deep Clustering for Unsupervised Learning of Visual Features*. https://doi.org/10.48550/arXiv.1807.05520

CatBoost Developers. *CatBoostClassifier | CatBoost*. Retrieved April 22, 2026, from https://catboost.ai/docs/en/concepts/python-reference_catboostclassifier

Cefis, M., & Carpita, M. (2025). A New xG Model for Football Analytics. *Journal of the Operational Research Society*, *76*(1), 1–13. https://doi.org/10.1080/01605682.2024.2323669

Cervone, D., D'Amour, A., Bornn, L., & Goldsberry, K. (2016). A Multiresolution Stochastic Process Model for Predicting Basketball Possession Outcomes. *Journal of the American Statistical Association*, *111*(514), 585–599. https://doi.org/10.1080/01621459.2016.1141685

Chen, S., & Guo, W. (2023). Auto-Encoders in Deep Learning—A Review with New Perspectives. *Mathematics*, *11*(8), 1777–1778. https://doi.org/10.3390/math11081777

Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. https://doi.org/10.1145/2939672.2939785

Coaches' Voice. (2024). *What Is Rest Defence? Football Tactics Explained*. https://learning.coachesvoice.com/cv/rest-defence-explained/

Cox, D. R. (1958). The Regression Analysis of Binary Sequences. *Journal of the Royal Statistical Society. Series B (Methodological)*, *20*(2), 215–242. https://www.jstor.org/stable/2983890

Davis, J., Decroos, T., & Robberechts, P. (2020, May 5). *Introducing Atomic-SPADL: A New Way to Represent Event Stream Data*. https://dtai.cs.kuleuven.be/sports/blog/introducing-atomic-spadl:-a-new-way-to-represent-event-stream-data/

Decroos, T., & Davis, J. (2020). Interpretable Prediction of Goals in Soccer. *Proceedings of the AAAI-20 Workshop on Artifical Intelligence in Team Sports*. AI in Team Sports Organising Committee. https://tomdecroos.github.io/reports/interpret_vaep.pdf

Decroos, T., Bransen, L., Van Haaren, J., & Davis, J. (2019). Actions Speak Louder than Goals: Valuing Player Actions in Soccer. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1851–1861. https://doi.org/10.1145/3292500.3330758

Decroos, T., Bransen, L., Van Haaren, J., & Davis, J. (2020). *VAEP: An Objective Approach to Valuing On-the-Ball Actions in Soccer (Extended Abstract)*. *5*, 4696–4700. https://doi.org/10.24963/ijcai.2020/648

DeLong, E. R., DeLong, D. M., & Clarke-Pearson, D. L. (1988). Comparing the Areas under Two or More Correlated Receiver Operating Characteristic Curves: A Nonparametric Approach. *Biometrics*, *44*(3), 837–845. https://doi.org/10.2307/2531595

Everett, G., Beal, R., Matthews, T., Norman, T., & Ramchurn, G. (2022). Contextual Expected Threat Using Spatial Event Data. *Statsbomb Conference 2022*. https://geverett.co.uk/pdfs/Gregory-Everett-et-al-Contextual-xT-using-Spatial-Event-Data.pdf

Fernandez, J., Bornn, L., & Cervone, D. (2021). A Framework for the Fine-Grained Evaluation of the Instantaneous Expected Value of Soccer Possessions. *Machine Learning*, *110*(6), 1389–1427. https://doi.org/10.1007/s10994-021-05989-6

Fernández, J., Bornn, L., & Fraser, S. (2018). *Wide Open Spaces: A Statistical Technique for Measuring Space Creation in Professional Soccer*. https://www.semanticscholar.org/paper/Wide-Open-Spaces%3A-A-statistical-technique-for-space-Fern%C3%A1ndez-Bornn/a9f6975e93f8c86dadb9249750712305511d5b69

Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, *29*(5), 1189–1232. https://www.jstor.org/stable/2699986

Goes, F., Meerhoff, L., Bueno, M., Rodrigues, D., Moura, F., Brink, M., Elferink-Gemser, M., Knobbe, A., Cunha, S., Torres, R., & Lemmink, K. (2021). Unlocking the Potential of Big Data to Support Tactical Performance Analysis in Professional Soccer: A Systematic Review. *European Journal of Sport Science*, *21*(4), 481–496. https://doi.org/10.1080/17461391.2020.1747552

Green, S. (2012). *Assessing The Performance of Premier League Goalscorers*. https://www.statsperform.com/resource/assessing-the-performance-of-premier-league-goalscorers/

Holm, S. (1979). A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, *6*(2), 65–70. https://www.jstor.org/stable/4615733

Jacobs, A. Z., & Wallach, H. (2021). Measurement and Fairness. *Proceedings of the 2021 ACM Conference on Fairness, Accountability, And Transparency*, 375–385. https://doi.org/10.1145/3442188.3445901

Jo, G., Kang, M., Lee, K., Lee, M., Bauer, P., & Ko, S.-K. (2026, January 29). *VERSA: Verified Event Data Format for Reliable Soccer Analytics*. https://doi.org/10.48550/arXiv.2601.21981

Kim, H., Choi, H.-J., Kim, C. J., Yoon, J., & Ko, S.-K. (2023). Ball Trajectory Inference from Multi-Agent Sports Contexts Using Set Transformer and Hierarchical Bi-LSTM. *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 4296–4307. https://doi.org/10.1145/3580305.3599779

Kuhn, H. W. (1955). The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*, *2*(1–2), 83–97. https://doi.org/10.1002/nav.3800020109

Lee, J., Lee, Y., Kim, J., Kosiorek, A. R., Choi, S., & Teh, Y. W. (2019, May 26). *Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks*. https://doi.org/10.48550/arXiv.1810.00825

Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., & Talwalkar, A. (2018, June 18). *Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization*. https://doi.org/10.48550/arXiv.1603.06560

Liu, D. C., & Nocedal, J. (1989). On the Limited Memory BFGS Method for Large Scale Optimization. *Mathematical Programming*, *45*(1), 503–528. https://doi.org/10.1007/BF01589116

Lumley, T., Diehr, P., Emerson, S., & Chen, L. (2002). The Importance of the Normality Assumption in Large Public Health Data Sets. *Annual Review of Public Health*, *23*, 151–169. https://doi.org/10.1146/annurev.publhealth.23.100901.140546

Macdonald, B. (2012). *An Expected Goals Model for Evaluating NHL Teams and Players*. https://www.semanticscholar.org/paper/An-Expected-Goals-Model-for-Evaluating-NHL-Teams-Macdonald/001dfd90cc14f6444f9c5745198137f4505a814b

McInnes, L., Healy, J., & Melville, J. (2020, September 18). *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction*. https://doi.org/10.48550/arXiv.1802.03426

Mead, J., O'Hare, A., & McMenemy, P. (2023). Expected Goals in Football: Improving Model Performance and Demonstrating Value. *PLOS ONE*, *18*(4), e282295. https://doi.org/10.1371/journal.pone.0282295

Murtagh, F., & Contreras, P. (2017). Algorithms for Hierarchical Clustering: An Overview, II. *Wires Data Mining and Knowledge Discovery*, *7*(6), e1219. https://doi.org/10.1002/widm.1219

Murtagh, F., & Legendre, P. (2014). Ward's Hierarchical Clustering Method: Clustering Criterion and Agglomerative Algorithm. *Journal of Classification*, *31*(3), 274–295. https://doi.org/10.1007/s00357-014-9161-z

Naeini, M. P., Cooper, G., & Hauskrecht, M. (2015). Obtaining Well Calibrated Probabilities Using Bayesian Binning. *Proceedings of the AAAI Conference on Artificial Intelligence*, *29*(1). https://doi.org/10.1609/aaai.v29i1.9602

Olthof, S., & Davis, J. (2025). Perspectives on Data Analytics for Gaining a Competitive Advantage in Football: Computational Approaches to Tactics. *Science and Medicine in Football*, *0*(0), 1–13. https://doi.org/10.1080/24733938.2025.2533784

Pappalardo, L., Cintia, P., Ferragina, P., Massucco, E., Pedreschi, D., & Giannotti, F. (2019). PlayeRank: Data-driven Performance Evaluation and Player Ranking in Soccer via a Machine Learning Approach. *ACM Transactions on Intelligent Systems and Technology (TIST)*, *10*(5), 1–27. https://doi.org/10.1145/3343172

Parzen, E. (1962). On Estimation of a Probability Density Function and Mode. *The Annals of Mathematical Statistics*, *33*(3), 1065–1076. https://www.jstor.org/stable/2237880

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-Learn: Machine Learning in Python. *J. Mach. Learn. Res.*, *12*(null), 2825–2830. https://dl.acm.org/doi/10.5555/1953048.2078195

Perktold, J., Seabold, S., Sheppard, K., ChadFulton, Shedden, K., jbrockmendel, grana6, Quackenbush, P., Arel-Bundock, V., McKinney, W., Langmore, I., Baker, B., Gommers, R., yogabonito, scherrer, Zhurko, Y., Brett, M., Giampieri, E., yl565, … Halchenko, Y. (2024, April 17). *Statsmodels/Statsmodels: Release 0.14.2*. https://doi.org/10.5281/ZENODO.593847

Peters, A. J., Parmar, N., Davies, M., Reeves, M., Sormaz, M., & James, N. (2024). Expected Pass Turnovers (xPT) - a Model to Analyse Turnovers from Passing Events in Football. *Journal of Sports Sciences*, *42*(11), 1002–1010. https://doi.org/10.1080/02640414.2024.2379697

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2019, January 20). *CatBoost: Unbiased Boosting with Categorical Features*. https://doi.org/10.48550/arXiv.1706.09516

Reep. *Reep - The Football Identity Register*. Retrieved August 12, 2026, from https://reep.football/

Robberechts, P., & Davis, J. (2020). *How Data Availability Affects the Ability to Learn Good xG Models* (pp. 17–27). https://doi.org/10.1007/978-3-030-64912-8_2

Rosenblatt, M. (1956). Remarks on Some Nonparametric Estimates of a Density Function. *The Annals of Mathematical Statistics*, *27*(3), 832–837. https://www.jstor.org/stable/2237390

Rousseeuw, P. (1987). Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. *Journal of Computational and Applied Mathematics*, *20*, 53–65. https://doi.org/10.1016/0377-0427%2887%2990125-7

Schaerlaeckens, L. (2025). The Belgian Lab Shaping Modern Soccer's Data Revolution. *The Guardian*. https://www.theguardian.com/football/2025/jun/03/soccer-analytics-jesse-davis-leuven

Schubert, E. (2023). Stop Using the Elbow Criterion for K-Means and How to Choose the Number of Clusters Instead. *ACM SIGKDD Explorations Newsletter*, *25*(1), 36–42. https://doi.org/10.1145/3606274.3606278

scikit-learn Developers. *RandomForestClassifier*. Retrieved April 22, 2026, from https://scikit-learn/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

Singh, K. (2018). *Introducing Expected Threat (xT)*. https://karun.in/blog/expected-threat.html

Socceraction Developers. *Socceraction 1.5.3 Documentation*. Retrieved April 22, 2026, from https://socceraction.readthedocs.io/en/latest/documentation/intro.html

Spearman, W., Basye, A., Dick, G., Hotovy, R., & Pop, P. (2017, March 3). *Physics-Based Modeling of Pass Probabilities in Soccer*. https://www.researchgate.net/publication/315166647_Physics-Based_Modeling_of_Pass_Probabilities_in_Soccer

StatsBomb. (2026a). *StatsBomb Open Data* [Computer software]. StatsBomb. https://github.com/statsbomb/open-data

StatsBomb. (2026b, July 6). *Statsbomb/Statsbombpy*. https://github.com/statsbomb/statsbombpy

Sá-Freire, L. M. de. (2025). Event Data Based Action Valuation: Possibilities and Limitations. In D. Memmert (Ed.), *Artificial Intelligence and Machine Learning in Sports Science* (pp. 285–292). Springer. https://doi.org/10.1007/978-3-662-70155-3_20

Transfermarkt. *Football Transfers, Rumours, Market Values and News*. Retrieved August 12, 2026, from https://www.transfermarkt.com/

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. https://doi.org/10.48550/arXiv.1706.03762

Wakelam, E., Steuber, V., & Wakelam, J. (2022). The Collection, Analysis and Exploitation of Footballer Attributes: A Systematic Review. *Journal of Sports Analytics*, *8*(1), 31–67. https://doi.org/10.3233/JSA-200554

Wales, D. J., & Doye, J. P. K. (1997). Global Optimization by Basin-Hopping and the Lowest Energy Structures of Lennard-Jones Clusters Containing up to 110 Atoms. *The Journal of Physical Chemistry a*, *101*(28), 5111–5116. https://doi.org/10.1021/jp970984n

Welch, B. L. (1951). On the Comparison of Several Mean Values: An Alternative Approach. *Biometrika*, *38*(3/4), 330–336. https://doi.org/10.2307/2332579

Wikipedia. (2026a). *Ballon d'Or*. https://en.wikipedia.org/w/index.php?title=Ballon_d%27Or&oldid=1365065329

Wikipedia. (2026b). *Ballon d'Or Féminin*. https://en.wikipedia.org/w/index.php?title=Ballon_d%27Or_F%C3%A9minin&oldid=1365090455

Wright, S. P. (1992). Adjusted P-Values for Simultaneous Inference. *Biometrics*, *48*(4), 1005–1013. https://doi.org/10.2307/2532694

Xie, E., Wang, W., Yu, Z., Anandkumar, A., Alvarez, J. M., & Luo, P. (2021, May 31). *SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers*. https://arxiv.org/abs/2105.15203v3